



Research article

Effects of feature selection and normalization on network intrusion detection

Mubarak Albarka Umar^{a,b,*}, Zhanfang Chen^b, Khaled Shuaib^a, Yan Liu^c^a College of Information Technology, United Arab Emirates University, Al Ain, 15551, United Arab Emirates^b School of Computer Science and Technology, Changchun University of Science and Technology, Changchun, 130022, China^c Department of Computer Science, Shantou University, Shantou, Guangdong, 515063, China

ARTICLE INFO

Keywords:

Cybersecurity
Intrusion detection system
Machine learning
Deep learning
Feature selection
Normalization

ABSTRACT

The rapid rise of cyberattacks and the gradual failure of traditional defense systems and approaches led to using artificial intelligence (AI) techniques (such as machine learning (ML) and deep learning (DL)) to build more efficient and reliable intrusion detection systems (IDSs). However, the advent of larger IDS datasets has negatively impacted the performance and computational complexity of AI-based IDSs. Many researchers used data preprocessing techniques such as feature selection and normalization to overcome such issues. While most of these researchers reported the success of these preprocessing techniques on a shallow level, very few studies have been performed on their effects on a wider scale. Furthermore, the performance of an IDS model is subject to not only the utilized preprocessing techniques but also the dataset and the ML/DL algorithm used, which most of the existing studies give little emphasis on. Thus, this study provides an in-depth analysis of feature selection and normalization effects on IDS models built using three IDS datasets: NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018, and various AI algorithms. A wrapper-based approach, which tends to give superior performance, and min-max normalization methods were used for feature selection and normalization, respectively. Numerous IDS models were implemented using the full and feature-selected copies of the datasets with and without normalization. The models were evaluated using popular evaluation metrics in IDS modeling, intra- and inter-model comparisons were performed between models and with state-of-the-art works. Random forest (RF) models performed better on NSL-KDD and UNSW-NB15 datasets with accuracies of 99.86% and 96.01%, respectively, whereas artificial neural network (ANN) achieved the best accuracy of 95.43% on the CSE-CIC-IDS2018 dataset. The RF models also achieved an excellent performance compared to recent works. The results show that normalization and feature selection positively affect IDS modeling. Furthermore, while feature selection benefits simpler algorithms (such as RF), normalization is more useful for complex algorithms like ANNs and deep neural networks (DNNs), and algorithms such as Naive Bayes are unsuitable for IDS modeling. The study also found that the UNSW-NB15 and CSE-CIC-IDS2018 datasets are more complex and more suitable for building and evaluating modern-day IDS than the NSL-KDD dataset. Our findings suggest that prioritizing robust algorithms like RF, alongside complex models such as ANN and DNN, can significantly enhance IDS performance. These insights provide valuable guidance for managers to develop more effective security measures by focusing on high detection rates and low false alert rates.

1. Introduction

As the Internet, networks, and computer systems play increasingly essential roles in our daily activities, they have become prime targets for cybercriminals. Consequently, it is imperative to explore optimal strategies to guarantee the security of our networks and systems. Initially, the first line of defense such as firewalls, user authentication, and data encryption is

used; however, they have demonstrated inadequacies. Due to their limitations, intrusion detection systems (IDSs) are now utilized to monitor computer and network intrusions actively. IDSs leverage distinctive analytical techniques to detect attacks, identify their sources, and promptly notify network administrators (Sowmya and Mary Anita, 2023).

The two primary methods used in IDSs are signature-based and anomaly-based detection. Although signature-based detection has

* Corresponding author. College of Information Technology, United Arab Emirates University, Al Ain, 15551, United Arab Emirates.

E-mail address: 700040629@uaeu.ac.ae (M.A. Umar).

Peer review under responsibility of Xi'an Jiaotong University.

<https://doi.org/10.1016/j.dsm.2024.08.001>

Received 29 February 2024; Received in revised form 19 July 2024; Accepted 6 August 2024

Available online 13 August 2024

2666-7649/© 2024 Xi'an Jiaotong University. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

dominated IDS use in practice, cybersecurity researchers are becoming more interested in anomaly-based IDS due to the ongoing emergence of zero-day and new types of intrusion attacks and the approach's inability to detect those novel attacks (Sowmya and Mary Anita, 2023). Artificial intelligence (AI) techniques such as machine learning (ML) and deep learning (DL) are commonly employed in building more reliable and effective IDS. To accurately detect an intrusion attack, the ML/DL techniques need to learn from experience—in this case, a dataset. However, in addition to lowering the performance of ML-based IDS models, the introduction of larger IDS datasets is also making the process more computationally complex. Many researchers use data preprocessing techniques like feature selection and normalization to get around these problems (Albulayhi et al., 2022; Bridges et al., 2019; Kasongo and Sun, 2020b).

While most of the existing IDS studies used AI algorithms and performed normalization and/or feature selection in building IDS models, few in-depth studies were conducted on the effects of these two preprocessing techniques. Additionally, these studies typically reported broad conclusions about the benefits of feature selection and/or normalization on the IDS models. However, this may not always be the case because the IDS model's performance is impacted not only by feature selection and normalization but also by the dataset and algorithms used, which most existing studies overlook. IDS models should therefore be created utilizing various datasets and algorithms to facilitate objective comparisons and a comprehensive deduction of the implications of feature selection and normalization in IDS modeling on the datasets and the algorithms. This study therefore sought to close this gap.

An in-depth study of the effects of feature selection and normalization is performed in this work. Six of the most used AI algorithms in IDS modeling (Özgür and Erdem, 2016) are selected. We chose three datasets—NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018—frequently used by researchers, with two of them recognized as contemporary IDS datasets (Leevy and Khoshgoftaar, 2020; Siddique et al., 2019). To assess the feature selection, a feature-selected copy of each dataset is made using an optimal wrapper-based feature selection approach with a decision tree algorithm as the feature evaluator. Wrapper-based feature selection evaluates feature subsets with a model, optimizing performance. While computationally expensive during offline modeling, this method ensures better model performance and generalization over others (Kumar et al., 2022; Umar et al., 2020). Further, this cost is incurred only once, not during online usage as depicted in Fig. 1, making it valuable for use with the selected datasets and algorithms. To evaluate the effect of normalization, min-max normalization, the predominantly used normalization method, often preferred over other methods like z-score normalization

due to its simplicity and robustness to outliers in IDS modeling is used to improve model effectiveness (Kumar et al., 2022). Using the six final datasets (i.e., the full and feature-selected copies), three different programs as summarized in Table 1 are implemented to build various models. The IDS models are evaluated and compared using well-known effective evaluation metrics in IDS modeling.

Fig. 1 depicts the model-building process, the built models are evaluated and compared offline before being deployed online for operation where live network traffic packets can be captured and analyzed to extract desirable features using tools such as Wireshark, Pcap, Nmap, ngrep, tshark, tcpdump, etc. (Alani et al., 2023), which can then be passed to the trained model for prediction to decide whether traffic is benign or an intrusion attack. The resulting decision can be used to secure the network by blocking the attack source and notifying the admin for additional measures. Algorithm 1 shows the online operation of the trained model in detecting intrusion.

Algorithm 1 Model's online operation

```

1: Input: Network traffic
2: Output: Intrusion detection alert
3:  $packet \leftarrow pcap(traffic)$ 
4:  $features \leftarrow tcpdump(packet)$ 
5:  $prediction \leftarrow trained\_model(features)$ 
6: if  $prediction$  is 'Anomaly' then
7:   Block( $source\_ip\_address$ )
8:   Notify users/admin
9: end if

```

In addition to the primary aim of this study, other important contributions are:

- Provides significant contributions towards advancing effective feature engineering techniques which are essential for constructing

Table 1
Intrusion detection system (IDS) programs implemented.

IDS program	Dataset features	Normalized	Total models built
Program A	All features	Yes	18
Program B	Selected features	Yes	18
Program C	Selected features	No	18

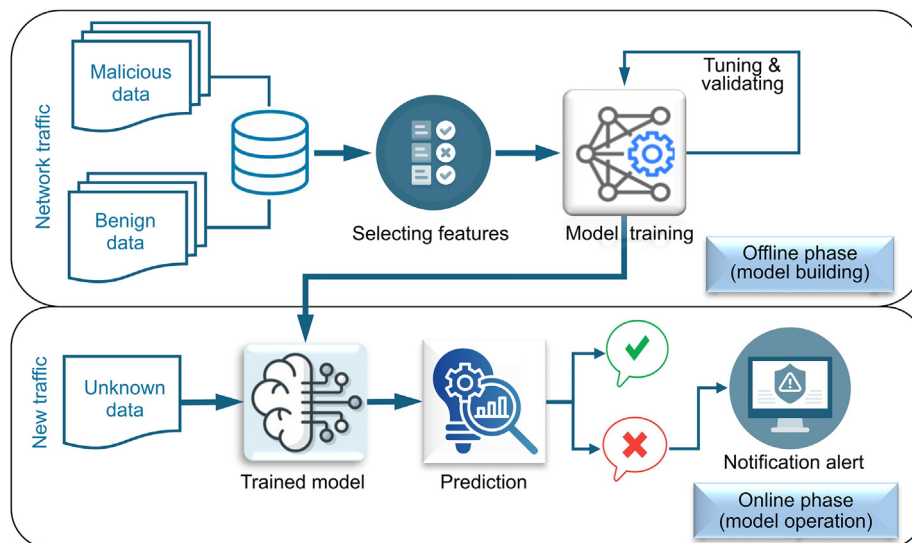


Fig. 1. Intrusion detection system (IDS) framework overview.

successful IDS models. These techniques including feature selection, feature normalization, and feature encoding, enhance the overall effectiveness of the models.

- A comprehensive analysis and utilization of various AI algorithms have been conducted to handle large volumes of network traffic data, resulting in a robust and efficient IDS model. Multiple model evaluation metrics were used to assess and compare the performance of these algorithms.
- Explores the impact of dataset choice on the performance of IDS by comparing the NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018 datasets. By assessing and verifying their effectiveness, reliability, and intricacy, the research contributes to a better understanding of how dataset and feature selection influence IDS performance.
- Effective feature selection on random forest (RF) yields a more robust IDS model. Support vector machine (SVM) requires normalization and incurs higher computational costs. Naive Bayes (NB) though quick to train, had the lowest performance, rendering them the least effective for IDS modeling. Additionally, artificial neural network (ANN) and deep neural network (DNN) models performed better with full datasets, indicating that feature selection is less critical for these models due to their capability to handle complex data.
- Provides various in-depth comparisons on several aspects such as feature selection, normalization, datasets, and IDS models. A comparison with the state-of-the-art works is also made.
- Our study contributes to the literature by demonstrating the importance of dataset selection, feature selection, and normalization in improving IDS performance. The findings offer practical insights for managers to optimize their security strategies by prioritizing robust algorithms such as RF and complex models like ANN and DNN, which can handle high-dimensional data and complex feature interactions.

The rest of the paper is organized as follows: Section 2 presents a review of the existing works, and their identified limitations. Section 3 explains some basic IDS concepts, ML, and the preprocessing techniques

used in this study. The experimental processes and tools used are presented in Section 4; the experimental results are presented and discussed in Section 5. Section 6 presents managerial implications. Section 7 concludes the study. Study limitations and directions for future research are identified in Section 8.

2. Literature review

This section presents some related IDS modeling studies and their drawbacks that utilize feature selection and/or normalization techniques in modeling IDS using various AI approaches. Table 2 gives an overview of the related studies.

2.1. Related works

Depren et al. (2005) proposed a hybrid IDS model using a self-organized map (SOM) for anomaly detection and a J48 tree for misuse detection on the KDDcup99 dataset. Six of 41 features were chosen for modeling without stating the utilized feature selection method. The selected features were normalized using the min-max method and used for training the model in WEKA software. The model demonstrated a detection rate (DR) of 99.90%. Wang et al. (2009) applied three algorithms (k-nearest neighbor (KNN), principal component analysis (PCA), and SVM) to model IDS on a normalized KDDcup99 dataset. They selected 34 numerical features and disregarded the remaining 7 nominal features. Using the three algorithms, they compared four attribute normalization methods on the dataset. Z-score outperforms the other normalization techniques on larger datasets based on false-positive and DRs.

Using a hybrid algorithm of supervised and unsupervised learning schemes on a non-zero normalized KDDcup99 dataset, Somwang and Lilakiatsakun (2011) proposed an anomaly-based IDS based on PCA and SVM. The PCA was used to select 10/41 features used to train the SVM. The performance metrics employed to assess the classifier were hit, miss,

Table 2
Summary of related works.

Reference	FS method (no. features)	Algorithm	Normalization	Dataset/ID approach	Evaluation metrics
Depren et al. (2005)	Not mentioned (6/41)	SOM/J48, DSS (WEKA)	Min-max (0–1)	KDD99/Hybrid	DR, FPR, and missed rate
Wang et al. (2009)	Not used, selected numeric features only (34/41)	PCA, KNN, SVM	Z-score, ordinal, min-max, and frequency	KDD99/Anomaly	Acc, DR, and FPR
Somwang and Lilakiatsakun (2011)	PCA (10/41)	SVM	Non-zero	KDD99/Anomaly	DR, FPR, miss, hit
Sivatha Sindhu et al. (2012)	GA (16/41)	Hybrid of neurotree	Min-max	NSL-KDD/Anomaly	TPR, FPR, PR, DR, F1
Song et al. (2013)	WEKA's CfsSubsetEval filter (8/41)	Fuzzy C/C4.5 (WEKA)	Min-max (0–1)	KDD99/Anomaly	TPR, FPR, P, TA, R, F1
Thaseen and Kumar (2013)	CFS & CONS filters (15/41)	Tree-based classifiers	Not mentioned	NSL-KDD/Anomaly	Acc, TPR, FPR, PR, DR, F1
Gotorlar et al. (2015)	Harmony search (21/41)	SVM (LibSvm)	Min-max (1–13)	NSL-KDD/Anomaly	DR, test time
Khammassi and Krichen (2017)	GA-LR KDD99 (18/41), UNSW-NB (20/42)	RF, C4.5, and NB tree	Log-scaling, min-max (0–1)	KDD99, UNSW-NB15/Anomaly	Acc, DR, FAR
Setiawan et al. (2019)	IG WEKA's filter (17/41)	SVM	Log-norm	NSL-KDD	Acc, sensitivity, specificity, FP, and TP
Khan et al. (2019)	DSAE (10/45)	Soft-max classifier	Min-max (0–1)	KDD99, UNSW-NB15/Anomaly	Acc, PR, DR, F1, and FAR
Wang et al. (2023)	Not used	DL models	Z-score	CSE-CIC-IDS2018/Anomaly	Acc, PR, DR, and F1
Du et al. (2023)	CNN	LSTM	Z-score	KDD99, NSL-KDD, UNSW-NB15/Anomaly	Acc, PR, DR, F1, and FPR
Lo et al. (2022)	Not mentioned	GNNs (E-GraphSAGE)	Not mentioned	ToN-IoT, BoT-IoT, NF-ToT-IoT, and NF-BoT-IoT/Anomaly	Acc, PR, DR, F1, and FPR
Cheng et al. (2021)	Not mentioned	GNNs (Alert-GCN)	Not mentioned	DARPA99 dataset alerts	Acc, PR, DR, and F1
This work	DT-based NSL-KDD (19/41) UNSW-NB (20/42), CIC-IDS2018 (12/77)	ANN, RF, SVM, KNN, NB, and DNN	Min-max (0–1)	NSL-KDD, UNSW-NB15, CSE-CIC-IDS2018/Anomaly	Acc, DR, FAR, and time

Note: FS: feature selection; DR: detection rate; FPR: false positive rate; PCA: principal component analysis; KNN: k-nearest neighbor; SVM: support vector machine; Acc: accuracy; GA: genetic algorithm; TPR: true positive rate; PR: precision; F1: F1-score; FAR: false alert rate; FP: false positive; TP: true positive; DL: deep learning; CNN: convolutional neural network; LSTM: long short-term memory; DT: decision tree; ANN: artificial neural network; RF: random forest; NB: Naive Bayes; DNN: deep neural network; GNN: graph neural network.

DR, and false positive rate (FPR). The experiment showed a DR of 97.4%; however, they recommend broader investigation. [Sivatha Sindhu et al. \(2012\)](#) presented a lightweight IDS for multi-class categorization that uses a hybrid of a neural network and decision tree (neurotree) for real classification and a wrapper-based genetic algorithm for feature selection. They used 16/41 features of NSL-KDD datasets and a min-max method to normalize the selected attributes. The models' performance was assessed using the WEKA evaluation metrics. Their proposed method achieved the highest DR of 98.38% in comparison to tree-based single classifiers.

An IDS modeling approach comprising a fuzzy C means clustering algorithm and a C4.5 decision tree algorithm was proposed by [Song et al. \(2013\)](#). They employed WEKA's CfsSubsetEval filter to select 8/41 features of the KDDcup99 dataset. The features were first normalized using the min-max technique, and then divided the training instances into clusters using the fuzzy C means clustering algorithm. Finally, each cluster was assigned a C4.5 algorithm for detecting anomaly instances on the test data. The method's performance was measured using six metrics, and WEKA was used to compare it to a C4.5 classifier with and without a feature selection algorithm. Their proposed method improves the performance results obtained by the C4.5 algorithm with only 19.5% of the total number of features. [Thaseen and Kumar \(2013\)](#) assessed the performance of six tree-based classifiers on the NSL-KDD dataset. They selected 15/41 features from the dataset using WEKA's CONS and CFS filters; however, no data normalization was carried out (perhaps because it does not affect the efficiency of tree-based algorithms ([James et al., 2021](#))). WEKA's evaluation metrics were employed to assess the models' performance with the randomtree model achieving the best accuracy and lowest false alert rate (FAR).

[Gotorlar et al. \(2015\)](#) proposed a harmony search-support vector machine (HS-SVM) method for intrusion detection on NSL-KDD datasets. They converted the nominal values to numeric and used the min-max method to normalize the numerical features after using harmony search to choose the 21/41 best features. The SVM model was trained using the LibSVM library. The proposed HS-SVM approach solves the SVM flaw of requiring a lot of time during testing based on the results of the model performance evaluation conducted using DR and test duration. In their study, [Setiawan et al. \(2019\)](#) presented an IDS model that combined SVM, normalization, and feature selection. They log-normalized numerical features after selecting 17/41 features from the NSL-KDD dataset using WEKA's modified rank-based information gain filter. Using WEKA's evaluation metrics, the model's overall accuracy was 99.8%.

[Khammassi and Krichen \(2017\)](#) proposed employing decision tree-based algorithms with a genetic algorithm-logistic regression wrapper selector (GALR-DT) to build IDS models. They utilized three DT-based classifiers: C4.5, RF, and NB tree. Using a genetic algorithm as a search strategy and logistic regression as a learning algorithm, they implemented a wrapper approach to determine the optimal subset of features on the KDDcup99 and UNSW-NB15 datasets. With KDDcup99, the GA-LR wrapper selected 18/41 features, and with UNSW-NB15, it selected 20/42 features. The data was normalized using log-scaling and the min-max of the 0–1 range. DR, accuracy, and FAR were used to compare the models' dataset-wise performance. Their results show that UNSW-NB15 provides the lowest FAR of 6.39% and a good classification accuracy compared to KDDcup99 and they concluded that the UNSW-NB15 dataset is more complex than the KDDcup99 dataset.

For effective network intrusion detection, [Khan et al. \(2019\)](#) presented a novel two-stage deep learning (TSDL) model based on a stacked auto-encoder with a soft-max classifier. The proposed model has two decision stages and can learn and classify useful feature representations in a semi-supervised approach. They evaluate the effectiveness of their methods on the KDDcup99 and UNSW-NB15 datasets. In each dataset, 10 features were chosen using DSAE feature selection and then normalized using the min-max technique. The proposed model performed well, with

high recognition rates of 99.996% and 89.134% for the KDDcup99 and UNSW-NB15 datasets, respectively. As older datasets like KDDcup99 and NSL-KDD are outdated and insufficient for detecting modern intrusion attacks, especially with DL models, [Wang et al. \(2023\)](#) proposed evaluating DL IDSs using the recent NID CSE-CIC-IDS2018 dataset. They employed and compared six deep learning models, including DNN, convolutional neural network (CNN), recurrent neural network (RNN), long short-term memory (LSTM), CNN + RNN, and CNN + LSTM for binary and multi-class classification tasks. The results showed high accuracy rates exceeding 98%. Interestingly, individual models showed eminent performance in terms of inference time compared to combined models, making them more suitable for implementation in IDS.

[Du et al. \(2023\)](#) proposed a DL-based model (NIDS-CNNLSTM) for NID in the Industrial Internet of Things (IIoT). The model combines LSTM's powerful learning capabilities with CNN's feature selection to classify network traffic, ensuring IIoT security. It was tested on KDDcup99, NSL-KDD, and UNSW-NB15 datasets, showing good convergence, high accuracy, and low training loss for binary and multi-class classification. Their proposed model significantly outperformed previous models with high accuracy, DR, and low FAR, making it suitable for large-scale, multi-scenario IIoT environments.

Recently, researchers are increasingly using graph representation learning to detect intrusions more effectively ([Bilot et al., 2023](#)). [Lo et al. \(2022\)](#) proposed E-GraphSAGE, a network intrusion detection system (NIDS) based on graph neural networks (GNNs) that utilizes graph-based data structures to capture edge features and topological information for detecting intrusions in IoT networks using flow-based data. Experimental evaluation on four recent NIDS benchmark datasets (ToN-IoT, BoT-IoT, NF-ToT-IoT, and NF-BoT-IoT) shows that E-GraphSAGE outperforms the state-of-the-art in terms of key classification metrics, demonstrating the potential of GNNs in NID, and encouraging further research. [Cheng et al. \(2021\)](#) proposed Alert-GCN, a framework based on GCN to correlate intrusion alerts and reveal attack scenarios. Using alerts generated from the DARPA99 dataset by Snort, Alert-GCN transforms these alerts into a graph with one-hot encoding and uses a two-layer GCN for node classification. Experimental results show that Alert-GCN outperforms traditional models in accuracy, precision, recall/DR, and F1-score for correlating alerts.

2.2. Limitations of related works

Our analysis of existing research reveals limitations in all reviewed works. The limitations can be grouped into three main categories: pre-processing (transformation and feature selection), modeling, and dataset issues.

- Preprocessing: Transformation (encoding, discretization/normalization), feature selection.
- Modeling stage: Problems selecting classifiers and rarely utilizing multiple classifiers.
- Dataset issues: Most studies use the KDDcup99 dataset and its variant NSL-KDD dataset, often without comparisons with other datasets; however, both datasets are considered outdated ([Leevy and Khoshgoftaar, 2020](#)).

2.2.1. Data encoding

Categorical features must be converted to numerical values for some AI algorithms to handle them. There are two types of categorical features: ordinal (ordered) and nominal (no particular order). The performance of many algorithms depends on how categorical features are encoded. One such nominal feature in the NSL-KDD is "Protocol_type", which has three values: UDP, TCP, and ICMP. Converting this attribute to a single numeric attribute using ordinal encoding introduces an ordering over the nominal values, this is an incorrect representation of the feature. Some

reviewed literature (Gotorlar et al., 2015; Khammassi and Krichen, 2017; Khan et al., 2019; Song et al., 2013) contains this error. A better solution is to use binary encoding or, better yet, one-hot (dummy) encoding, which maps each category to a vector containing 1 and 0 indicating the presence or absence of the feature's value.

2.2.2. Data discretization and normalization

Although discretization of numerical features plays an important role in data preprocessing (Ramírez-Gallego et al., 2016), it typically results in loss of information unlike normalization (Jin et al., 2007). Normalization is a crucial data preprocessing step that can enhance the accuracy and efficiency of classification algorithms, particularly for IDS models with large datasets (Azizjon et al., 2020; Kasongo and Sun, 2020b; Wang et al., 2009). While both methods can be utilized, normalization is essential in IDS cases where the data spans a large range of traffic values, and discretization by itself is not recommended because the value range is not as important. However, some studies opt for discretization over normalization (Thaseen and Kumar, 2013). Moreover, most reviewed studies place minimal emphasis on the effects of normalization on IDS model performance.

2.2.3. Feature selection

Owing to the large size and high dimensionality of IDS datasets, dimensionality reduction techniques are widely used to minimize the dimension of datasets and identify the best subset of features. As a result, IDS models perform better and require less computing time and resources (Taher et al., 2019). The subsequent section explains three fundamental feature selection techniques. However, only two of these techniques were primarily used in IDS modeling; the filter method, which is typically used in most reviewed studies, ignores the effects of the chosen feature subset on the IDS model's performance (Umar et al., 2021). In contrast, the wrapper method, though computationally expensive, results in better performance for the predefined classifier. Furthermore, some studies hand-pick specific features without using any feature selection methods, which may result in the removal of influential features (Depren et al., 2005; Wang et al., 2009).

2.2.4. Modeling stage

Exploring different algorithms and techniques can help to develop an accurate and reliable IDS model (Somwang and Lilakiatsakun, 2011). Nonetheless, most reviewed studies employ no more than three algorithms. Furthermore, some studies (Khammassi and Krichen, 2017; Thaseen and Kumar, 2013) overuse tree-based algorithms without verifying their effectiveness against alternative algorithms. Unfortunately, some regression and classification algorithms outperform tree-based algorithms in predictive accuracy, such as ensemble trees like a RF (James et al., 2021).

2.2.5. Dataset issues

KDDcup99 and its variant NSL-KDD are the commonly used datasets in IDS academic research (Özgür and Erdem, 2016; Siddique et al., 2019), and were used in most reviewed literature. However, these datasets generally lack modern-day attacks (Bridges et al., 2019; Khraisat et al., 2019). Using an up-to-date ID dataset is necessary for developing reliable and accurate IDS models in the current environment of continuously emerging new threats. Some contemporary chosen datasets appropriate for general network intrusion detection modeling and evaluation were suggested by some researchers (Creech and Hu, 2013; Haider et al., 2017; Moustafa and Slay, 2015; Ring et al., 2019). A better and more trustworthy IDS model can be built using the suggested datasets which are freely accessible. In addition, Ring et al. (2019) recommended utilizing multiple datasets with at least one publicly available dataset for reproducibility and general evaluation and to prevent building an overfitted IDS model with a single dataset. However, most studies only used one dataset.

In this study, we tackled these limitations by adopting specific strategies. Since most AI algorithms consider all features simultaneously

during training, we utilize one-hot encoding, which is well-suited for such algorithms (Cerdeira et al., 2018). Additionally, we applied min-max normalization, a commonly used method due to its simplicity and robustness to outliers (Kumar et al., 2022). We selected six widely used algorithms in IDS modeling (Özgür and Erdem, 2016). To ensure a comprehensive evaluation, we chose three datasets: one considered old (NSL-KDD) and the others current (UNSW-NB15, CSE-CIC-IDS2018) (Siddique et al., 2019). To optimize feature selection, we employ a decision tree wrapper-based approach for better model performance and generalization. Ultimately, we developed and evaluated various IDS models.

3. Basic theory and related knowledge

3.1. IDS

An IDS is a software program or device that monitors network and system traffic for malicious behavior, policy violations, and the presence of known threats, sending alarms when such things are detected. IDSs are security tools that are meant to improve the security of information and communication systems, much like firewalls, antivirus programs, and access control systems (Keshk et al., 2023). An IDS can be classified in two ways: based on data source/location and detection approach (Alkasasbeh and Al-Haj Baddar, 2023). The two most well-known classifications based on the data source are NIDSs and host intrusion detection systems (HIDSs). HIDS examines files and activities on the host computers in its most basic form, while NIDS monitors network traffic. The most popular detection methods are misuse-based (identifying registered bad patterns) and anomaly-based (identifying deviations from a model of "good" traffic, frequently utilizing AI techniques) (Lakshminarayana et al., 2019); the former is limited to identifying known attack types, while the latter often produces false positive alerts. Owing to these two methods' complementary qualities, a hybrid strategy combining them is frequently employed (Ayyagari et al., 2021). Nowadays, a lot of research has been done on creating a wide range of automated, fast, and effective IDSs with the help of AI techniques, advanced statistical learning, and expert-crafted rules (Bridges et al., 2019; Keshk et al., 2023).

3.2. AI

AI techniques (such as ML and DL) are the most popular approaches for creating IDSs (Özgür and Erdem, 2016). The AI techniques work by creating an explicit or implicit model that enables the recognition of patterns within raw data. Single, hybrid, or ensemble classifiers can be used when applying AI techniques to IDSs. Three operating modes for the employed classifier are supervised, unsupervised, and semi-supervised. Generally, the supervised mode performs better than the other modes (Albulayhi et al., 2022; Keshk et al., 2023). ANNs, KNN, NB, genetic algorithms, SVMs, logistic regression, DNNs, and decision trees are a few of the AI algorithms used in IDSs. Four fundamental steps are involved in developing an AI model: gathering data, preprocessing the data, choosing and training the model, and evaluating the model (Buczak and Guven, 2016) as depicted in Fig. 2. Among the numerous tasks that can be performed in the data preprocessing step are feature selection and normalization, two crucial concepts in this work.

3.2.1. Feature selection

This is a data reduction technique that involves selecting a subset of relevant features for model building without changing their dimensions. Feature selection shortens the training time of a model, decreases the likelihood of overfitting, and increases classification accuracy and generalization. Feature selection approaches can be classified into three types: (a) filters that extract features from data without the need for a learning algorithm; (b) wrappers that use a learning algorithm to identify useful features; and (c) embedded techniques that combine the two

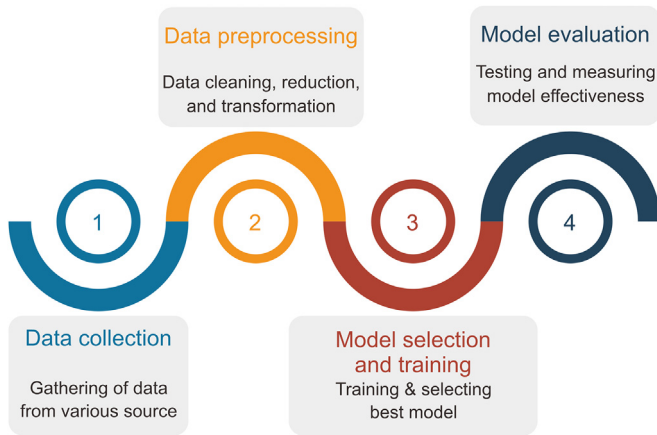


Fig. 2. Artificial intelligence (AI) basic modeling steps.

former methods with a classifier (Umar et al., 2020). In this work, the wrapper method is used due to its ability to optimize performance. The wrapper method necessitates a search strategy, direction, objective function, termination condition, and outcome evaluation which are discussed in Section 4.3.

3.2.2. Normalization

This is a data transformation technique that converts a dataset's wide-range numeric values to a common scale without distorting the range of the values. Data normalization aims to assign equal weight to each attribute. For classification algorithms utilizing neural networks or distance measurements, like nearest-neighbor and clustering algorithms, normalization is especially helpful in accelerating the model training phase (Kumar et al., 2022). While there are numerous ways to normalize data, the most popular ones are decimal scaling, z-score normalization, and min-max normalization (Han et al., 2023). Min-max normalization due to its simplicity and robustness to outliers in IDS modeling (Kumar et al., 2022) is utilized in this work to improve model effectiveness, its formula is as follows:

$$x_{\text{new}} = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (1)$$

3.3. Selected algorithms

We selected six of the most effective and widely used AI algorithms for IDSs (Özgür and Erdem, 2016). The algorithms are explained in the following.

3.3.1. SVM

An SVM is a supervised ML algorithm that determines the best hyperplane in a high-dimensional space for distinguishing instances of different classes. Fig. 3 depicts components of SVM. The key to SVM lies in its hyperplane's decision function $f(x)$ which can be mathematically stated as in Eq. (2).

$$f(x) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b) \quad (2)$$

where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the input vector, and b is the bias term (Cortes and Vapnik, 1995). The expected class of the instance is determined by the function in Eq. (2), which returns +1 in the case of a positive result and -1 in the case of a negative result. The distance between the support vectors (the nearest data points) and the hyperplane (decision boundary) is captured by the margin (γ) in Eq. (3).

$$\gamma = \frac{2}{\|\mathbf{w}\|} \quad (3)$$

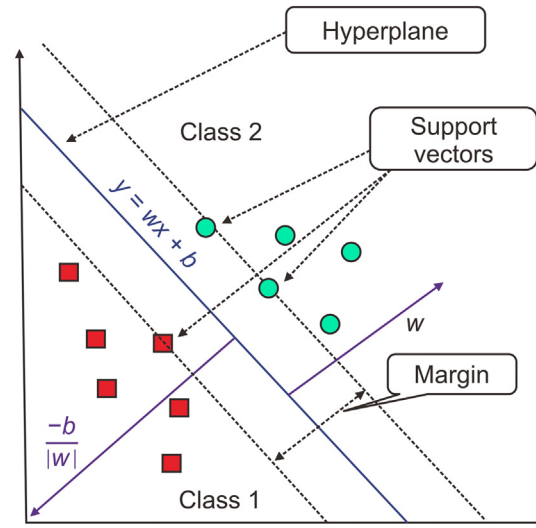


Fig. 3. Support vector machine (SVM) components. Reprinted from Chapter 6 - Machine learning for soil moisture assessment (Rani et al., 2022), Copyright (2022), with permission from Elsevier..

The aim is to find an optimal hyperplane that effectively separates the classes with the maximum margin. The kernel function improves the SVM's ability to handle complex relationships (Cortes and Vapnik, 1995). In this study, we used a support vector classifier based on LibSVM with the Radial Basis Function (RBF) kernel, implemented using Scikit-learn.

3.3.2. KNN

The KNN algorithm is one of the simplest and most popular supervised ML algorithms that makes classification or regression predictions of individual data points based on the majority class of its k closest data points in the feature space (Zhang, 2016). Fig. 4 shows how the choice of k can affect the classification of an unknown instance. Mathematically, KNN can be defined as:

$$f(x) = \arg \max_{y \in Y} \sum_{i \in N_k(x)} I(y = y_i) \quad (4)$$

where $f(x)$ is the prediction for the new instance x , Y is the set of possible classes, $N_k(x)$ is the set of KNNs to x , and $I(y = y_i)$ is an indicator function equal to 1 if the class of the i -th nearest neighbor is y and 0 otherwise.

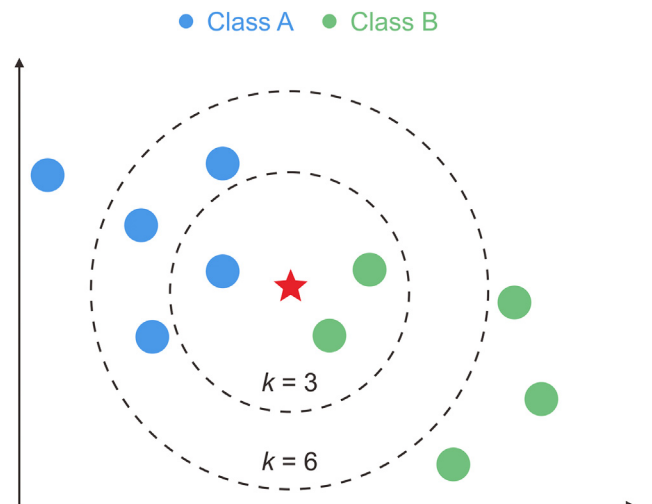


Fig. 4. K-nearest neighbors (KNN) classification (Surya Prasath et al., 2017).

3.3.3. RF

RF is a supervised ML algorithm that uses an ensemble learning technique to build a forest of decision trees at training time and merge their predictions through majority voting (for classification tasks) or averaging (for regression tasks) (Biau and Scornet, 2016). Each decision tree in the forest is trained on a different subset of training data. The final prediction $f(x)$ is given by:

$$f(x) = \frac{1}{N} \sum_{i=1}^N T_i(x) \quad (5)$$

where N is the number of decision trees in the forest, and $T_i(x)$ represents the prediction of the i -th decision tree for input x . RFs use bagging ensembling methods to improve accuracy and robustness to noise and outliers, as well as reduce overfitting (Biau and Scornet, 2016).

3.3.4. NB

An NB classifier is a probabilistic ML algorithm inspired by Bayes' theorem with a simple assumption of independence among features. NB calculates the probability of each class given the input features and selects the class with the highest probability as the predicted class. Despite its simplicity, NB works well with both small and large data due to its efficient training process rivaling more complex algorithms when set up correctly. It is computationally efficient as it takes linear time. It is commonly used for classification tasks such as text categorization and spam filtering (Yadav et al., 2020). Mathematically, the NB classifier calculates the probability of a class C_k given some input features $x_1, x_2, \dots, x_n$ using Bayes' theorem:

$$P(C_k | x_1, x_2, \dots, x_n) = \frac{P(C_k) \cdot P(x_1, x_2, \dots, x_n | C_k)}{P(x_1, x_2, \dots, x_n)} \quad (6)$$

The “naive” assumption of feature independence simplifies the conditional probability term to:

$$P(x_1, x_2, \dots, x_n | C_k) = P(x_1 | C_k) \cdot P(x_2 | C_k) \cdot \dots \cdot P(x_n | C_k) \quad (7)$$

These probabilities are estimated from the training data, often using techniques like Maximum Likelihood Estimation (MLE) or Laplace smoothing to tackle the problem of zero probability from the training set.

3.3.5. ANN

An ANN is a computational model inspired by the neural networks in the human brain. It consists of interconnected nodes (neurons) with associated weights that can be adjusted during training to learn patterns and relationships in the data. ANNs perform tasks by learning from input data without relying on explicit rules. They use activation functions to produce outputs, which allows them to handle complex, non-linear relationships in the data (Hagan et al., 2014). ANNs are suitable for various classification and regression tasks across diverse domains due to their ability to learn complex patterns. They are particularly used in IDS for their flexibility and adaptability to changing environments (Keshk et al., 2023). In this work, we use a fully connected feedforward neural network, also known as a multilayer perceptron (MLP), with three layers (input, hidden, and output). Mathematically, an ANN can be expressed as:

$$Y_i^l = f \left(\sum_{k=1}^J w_{i,k}^l x_k^{l-1} + b_i^l \right) \quad (8)$$

where Y_i^l is the output of unit i in layer l , x_k^{l-1} is the output from unit k in the previous layer, J is the number of units in the previous layer, b_i^l is the bias term for unit i , $w_{i,k}^l$ is the weight connecting unit k in the previous layer to unit i , and f is a non-linear activation function.

3.3.6. DNN

A DNN is a type of neural network that includes multiple layers between the input and output layers, known as hidden layers. DNNs are typically feedforward networks where data flows in one direction, from the input layer to the output layer. While ANNs usually consist of one or two hidden layers, DNNs have more than two hidden layers, allowing them to model more complex patterns (Hagan et al., 2014).

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (9)$$

DNNs, like ANNs, are inspired by biological neurons and compute the sum of inputs, applying an activation function at each layer before producing an output. This process is crucial for learning and approximating complex, non-linear relationships in real-world problems. The additional layers in DNNs enable them to learn hierarchical representations of data (Burkov, 2019). In this study, we used the Sigmoid activation function, defined in Eq. (9) in the output layer and the ReLU activation function in the three hidden layers. We used the binary cross-entropy loss function to measure the loss of sample using Eq. (10).

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)] \quad (10)$$

where $\hat{y}_i$ is the i -th predicted value, y_i is the corresponding true value, and N is the number of predicted values.

4. Methodology

This section explains the methodology for the experiment based on the four fundamental AI steps discussed. Besides summarizing the experimental tools employed, we also provide the pseudocodes of the various preprocessing and modeling aspects.

4.1. Experimental tools

Several tools are used in the IDS literature to implement, assess, and compare IDS models. The most popular tools are WEKA, general-purpose programming languages (like Java, Python, etc.), and MATLAB (Özgür and Erdem, 2016). In this work, Excel, WEKA, and Python are used for data analysis and exploration, preprocessing, as well as in implementing and validating the IDS models, respectively. All the experiments were executed using Python on an Ubuntu 16.04 LTS. All classical ML algorithms were implemented using Scikit-learn. The DNN was implemented using TensorFlow as the backend with Keras higher level framework. The CPU of the OS had a configuration (274.88 GB RAM, 500 GB hard disk, Intel(R) Xeon(R) Silver 4116 CPU @ 2.10 GHz x46). Other Python libraries used include NumPy, Pandas, Matplotlib, Scikit-learn, and so on. All the programs were implemented and executed in the same environment.

4.2. Dataset acquisition

4.2.1. UNSW-NB15 dataset

In 2015, the Australian Center for Cyber Security (ACCS) produced a new IDS dataset called UNSW-NB15. Using the IXIA Perfect Storm tool and a tcpdump tool, approximately 2.5 million samples, or 100 GB of raw data, were collected from contemporary network traffic, encompassing both normal and attack behaviors defined by 49 features. The generated features can be divided into five groups: additional generated features, flow features, basic features, content features, and time features. Nine distinct contemporary attack types (five more than in NSL-KDD) are included in the dataset, these include backdoor, denial-of-service (DoS), generic, reconnaissance, analysis, fuzzers, exploit, shellcode, and worm attacks (Moustafa and Slay, 2015). The UNSW-NB15 includes predefined splits of a training set (175,341 samples) and a testing set (82,332

samples) for ease of use and work reproducibility (Moustafa, 2015), both the training and testing set that are publicly available contain only 44 features, having 42 attributes and 2 classes. In this work, the training and testing of models are conducted using only the training set. Table 3 displays the broad distribution of total attacks (anomaly) and normal traffic samples of the training set used, which is appropriate given that binary classification is our primary focus.

4.2.2. NSL-KDD dataset

The KDDcup99 dataset is the data set used for the Third International Knowledge Discovery and Data Mining Tools Competition, which took place in 1999 alongside KDD-99, the Fifth International Conference on Knowledge Discovery and Data Mining. The dataset contains over 5 million training samples and over 2 million testing samples. Additionally, it has numerous redundant samples and imbalance classes (Tavallaee et al., 2009). The KDDcup99 dataset has been optimized and redesigned as the NSL-KDD (UNB, 2009). It eliminates duplicate records and offers logical and varied samples for the training and testing sets. The NSL-KDD dataset has 41 features, 3 categorical features, and 38 numerical features, just like the KDDcup99. The dataset includes four distinct attack types: probe, user-to-root (U2R), root-to-local (R2L), and DoS. It is speculated that the NSL-KDD dataset is not up to date (Bridges et al., 2019). The NSL-KDD is divided into two sets: a testing set comprising 22,544 samples (KDDTest+) and a training set comprising 125,973 samples (KDDTrain+). In this work, model training and testing are conducted using only the training set (KDDTrain+). The distribution of all attacks (anomaly) and normal traffic in the training set is summarized in Table 4.

4.2.3. CSE-CIC-IDS2018 dataset

The CSE-CIC-IDS2018 dataset, a successor to the CIC-IDS2017 dataset, is collaboratively developed by the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC) for evaluation of intrusion detection research and serves as a benchmark for IDS assessment due to its replication of complex network environments (Leevy and Khoshgoftaar, 2020). Captured over ten days, the dataset has 80 features and 15 attack types classifiable in seven different attack scenarios: brute-force, heartbleed, botnet, DoS, DDoS, web attacks, and infiltration of the network from inside. The attacking infrastructure consists of 50 machines, while the victim organization consists of 5 departments, 420 machines, and 30 servers. The dataset contains network traffic and system logs from each machine, in addition to 80 features extracted from the captured traffic using CICFlowMeter-V3 (UNB, 2018). Besides, the dataset is annotated with labels indicating the presence of attacks or normal network behavior, enabling supervised learning approaches for intrusion detection (Sharafaldin et al., 2018).

We used the semi-preprocessed and cleansed CSE-CIC-IDS2018 data from Kaggle (D'hooge, 2022) in which 80 traffic features were extracted using CICFlowmeter, the metadata features (such as flow ID, IP addresses, timestamps, etc.) were removed to avoid them acting as powerful shortcut predictors. (D'hooge et al., 2022). Missing, duplicates, and NaN records were also removed, and the data types of the dataset were downsized to save storage and improve loading times. Due to the huge size of the dataset in comparison to NSL-KDD and UNSW-NB15, and hardware limitations, we utilized only 10% of the extensive CSE-CIC-IDS2018 dataset for training and testing in our study to facilitate fair comparisons. The process of selecting the 10% is explained in the data reduction subsection of data preprocessing (Section 4.3). The distribution of all attacks (anomaly) and normal (benign) traffic is

Table 3
UNSW-NB15 distribution sample.

Category	Sample size	Distribution (%)
Total attacks	119,341	68.06
Normal	56,000	31.94
Overall samples	175,341	100

Table 4
NSL-KDD distribution sample.

Category	Sample size	Distribution (%)
Total attacks	58,630	46.54
Normal	67,343	53.46
Overall samples	125,973	100

Table 5
CSE-CIC-IDS2018 distribution sample.

Traffic type (class labels)	Number of samples	Selected samples
DDoS attacks-LOIC-HTTP	575,364	139,857
DDoS attack-HOIC	198,861	48,338
DoS attacks-Hulk	145,199	35,294
Bot	144,535	35,133
Infiltration	118,483	28,800
SSH-Bruteforce	94,048	22,860
DoS attacks-GoldenEye	41,406	10,064
DoS attacks-Slowloris	9,908	9,908
DDoS attack-LOIC-UDP	1,730	1,730
Brute force -Web	568	568
Brute force -XSS	229	229
SQL injection	85	85
DoS attacks-SlowHTTPTest	55	55
FTP-BruteForce	53	53
Anomaly (all attack types)	1,330,524	332,974 (total 5% of 10%)
Normal (benign)	5,329,008	332,976 (5% of 10%)

summarized in Table 5.

4.3. Data preprocessing

This study incorporates two primary preprocessing steps: data reduction (filtration and feature selection), and data transformation (data normalization and encoding). The preprocessing operations performed on the NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018 datasets for data reduction (feature selection) and data transformation is shown in algorithms 2 and 3, respectively. The algorithm 2 does not include data filtration as it is performed manually.

4.3.1. Data reduction

4.3.1.1. Data filtration. To minimize computation time and prepare the data for feature selection, irrelevant data are removed. There are 42 attributes in the UNSW-NB15 dataset, along with two class attributes and an extra id attribute (which is removed). Because we are only interested in binary classification, the class attribute `attack_cat` which lists the categories of attacks and the normal state is also eliminated before feature selection is performed. No filtration is performed on the NSL-KDD dataset as all the data features are relevant. We selected only 10% of the CSE-CIC-IDS2018 dataset, composed of 5% normal (benign) class, and the remaining 5% instances are made from a combination of all attack classes with less than 10,000 samples and an appropriate percentage (23.4% of 5%) of samples from attack classes with greater than 10,000 samples. As we are interested in binary classification, we merged all attack types as “anomaly” while balancing the ratio of normal and anomaly. The final 10% of the dataset extracted is available on Kaggle¹ for reproducibility.

The three datasets are split into train and test sets of distinct samples in the proportions of 1/3 (33.3%) and 2/3 (66.7%), respectively. WEKA's unsupervised instance resample filter is used to guarantee balanced data splitting to prevent building overfitted models that might perform poorly on out-of-sample data (Witten et al., 2016).

¹ <https://www.kaggle.com/code/albarka/generating-10-of-cic-ids2018-dataset>.

4.3.1.2. Feature selection. In feature selection, it is crucial to prevent information leakage and the ensuing creation of misleading models (Witten et al., 2016). Thus, we used only the training set in feature selection, and the testing set was used for performance evaluation to guarantee the creation of trustworthy models that can identify intrusion attacks in operational settings.

In this study, we adopted the wrapper-based feature selection approach with a decision tree algorithm as the feature evaluator, as illustrated in Fig. 5. The utilized decision tree algorithm is the J48, a Java implementation of Quinlan’s C4.5 (Salzberg, 1994). WEKA’s implementation of feature selection is used (Witten et al., 2016). The Best First Forward search strategy is employed in feature search, with accuracy as the evaluation metric and five consecutive non-improving nodes serving as the search-stopping criterion. The pseudocode of the feature selection steps used is displayed in Algorithm 2. The algorithm iteratively adds features to the feature set based on their performance in improving accuracy.

Algorithm 2 Wrapper-based DT feature selection

```

1: Input: Filtered dataset  $D$  with  $n$  features, Number of folds  $F$ 
2: Output: Dataset  $D'$  with  $m$  features
3: Initialize:  $D' \leftarrow \emptyset$ ,  $m \leftarrow 0$ ,  $best\_accuracy \leftarrow 0$ ,  $non\_improving \leftarrow 0$ 
4: while  $non\_improving < 5$  do
5:   for each feature  $f_i \in D$  not in  $D'$  do
6:      $D'' \leftarrow D \cup \{f_i\}$ 
7:      $accuracy \leftarrow CrossValidation(D'', F, DT)$ 
8:     if  $accuracy > best\_accuracy$  then
9:        $best\_accuracy \leftarrow accuracy$ ,  $f_{best} \leftarrow f_i$ 
10:    end if
11:  end for
12:  if  $best\_accuracy > 0$  then
13:     $D' \leftarrow D' \cup \{f_{best}\}$ ,  $m \leftarrow m + 1$ ,  $non\_improving \leftarrow 0$ 
14:  else
15:     $non\_improving \leftarrow non\_improving + 1$ 
16:  end if
17:   $D \leftarrow D - \{f_{best}\}$ 
18: end while
19: Return  $m$ ,  $D'$ 

```

Following feature selection, the best number of features for UNSW-NB15, NSL-KDD, and CSE-CIC-IDS2018 were twenty, nineteen, and twelve, respectively. The feature subsets were gathered using the supervised attribute Remove filter in WEKA. As a result, three more datasets are derived, increasing the total number of datasets to six: three feature-selected versions of NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018, and the three full datasets. Table 6 shows the selected optimal features.

4.3.2. Data transformation

4.3.2.1. Data normalization. The datasets comprised of numerical and nominal features. Numerical features of all datasets in Program A & B are normalized using min-max normalization (Eq. (1)) to prevent classifier

bias towards numeric features with a wide range of values. The normalization is done after feature selection to avoid interfering with the feature selection process.

Algorithm 3 Data preprocessing (transformation)

```

1: Input: Dataset  $D$ 
2: Output: Preprocessed dataset  $D'$ 
3: Initialize:  $D' \leftarrow \emptyset$ 
4: for each program  $p \in \{A, B, C\}$  do
5:    $F_p^{num} \leftarrow$  Numerical features in  $D$ 
6:    $F_p^{cat} \leftarrow$  Categorical features in  $D$ 
7:   for each feature  $f \in F_p^{num}$  do
8:     if  $p \neq C$  then
9:        $f' \leftarrow MinMaxScaler(f, D)$ 
10:     $D' \leftarrow D' \cup f'$ 
11:   end if
12: end for
13: for each feature  $f \in F_p^{cat}$  do
14:    $f' \leftarrow OneHotEncoder(f, D)$ 
15:    $D' \leftarrow D' \cup f'$ 
16: end for
17: end for
18: Return  $D'$ 

```

4.3.2.2. Data encoding. Throughout the datasets, every categorical (nominal) feature is one-hot encoded. Three nominal features (protocol_type, service, and flag) of NSL-KDD are one-hot encoded. The protocol_type feature is encoded from (a) to (b) as demonstrated in Table 7. Following this process, the 41-dimensional features are mapped into 122-dimensional features, of which 84 have encoded binary values of the three categorical features (protocol_type, service, and flag) and 38 are numeric. The NSL-KDD feature-selected version has just two nominal features (service and flag) that are correspondingly encoded. Similarly, the UNSW-NB15 and its feature-selected version were all one-hot encoded for the three nominal features (proto, service, and state). Due to the absence of nominal features, no encoding is performed on the CSE-CIC-IDS2018.

The encoding is done after feature selection and normalization to avoid the loss of one-hot encoded features during the feature selection process. Algorithm 3 displayed the transformation performed in pre-processing the datasets across the three programs. Only the final encoded features with the feature dimensions as depicted in Table 8 (plus target feature) are used in building and assessing the models.

4.4. Model selection and training

Model selection entails selecting the most appropriate model from a collection of candidate models (Hastie et al., 2009). The datasets are divided into two sets, the training and the testing sets for the modeling. The building of the models consists of two stages: the training and testing phases. In the training stage, the training set is used to train the algorithms on a 10-fold GridSearchCV method to build several models and select the statistically optimal model based on training accuracy ($AccTr$) as shown in Algorithm 4. Table 9 depicts the final parameters of the

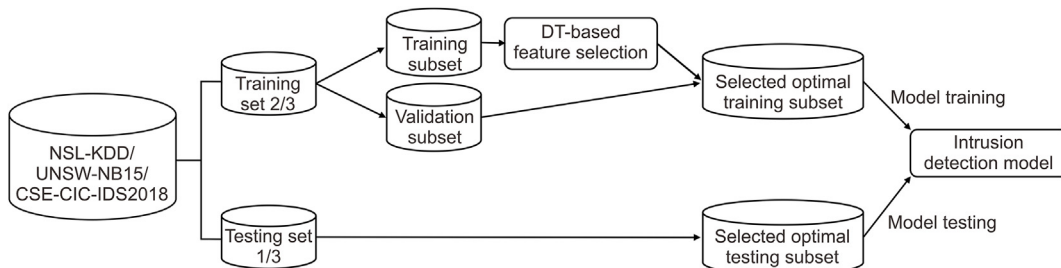


Fig. 5. Decision tree (DT) wrapper-based feature selection.

Table 6
Selected optimal features.

No.	UNSW-NB15	No.	NSL-KDD	No.	CSE-CIC-IDS2018
2	proto	1	duration	3	Total Fwd Packets
3	service	3	service	5	Fwd Packets Length Total
4	state	4	flag	11	Bwd Packet Length Max
5	spkts	5	src_bytes	16	Flow Packets/s
7	sbytes	6	dst_bytes	22	Fwd IAT Mean
8	dbytes	11	num_failed_logins	26	Bwd IAT Total
11	dttl	14	root_shell	27	Bwd IAT Mean
14	sloss	17	num_file_creations	46	RST Flag Count
15	dloss	23	count	49	URG Flag Count
17	dinpkt	24	srv_count	66	Init Fwd Win Bytes
18	sjit	25	serror_rate	69	Fwd Seg Size Min
27	smean	26	srv_error_rate	76	Idle Max
31	ct_srv_src	27	rerror_rate		
32	ct_state_ttl	32	dst_host_count		
33	ct_dst_ltm	34	dst_host_same_srv_rate		
34	ct_src_dport_ltm	35	dst_host_diff_srv_rate		
36	ct_dst_src_ltm	38	dst_host_serror_rate		
39	ct_flw_http_mthd	39	dst_host_srv_error_rate		
40	ct_src_ltm	40	dst_host_rerror_rate		
41	ct_srv_dst				

Table 7
One-hot encoding example.

(a)	(b)		
Protocol_type	UDP	TCP	ICMP
UDP	1	0	0
TCP	0	1	0
ICMP	0	0	1

Table 8
Dataset feature dimensions.

One-hot encoding	UNSW-NB15		NSL-KDD		CES-CIC-IDS2018	
	All	FS	All	FS	All	FS
Before encoding	42	20	41	19	77	12
After encoding	194	172	122	98	77	12

Table 9
Final parameters for the algorithms.

Algorithm	Parameters
NB (GaussianNB (·))	var_smoothing = 1e-09, priors = None (all classes are assumed to be equally likely)
RF (RandomForestClassifier (·))	n_estimators = 100, criterion = 'gini', max_depth = None, min_samples_split = 2, min_samples_leaf = 1, bootstrap = True, class_weight = None
KNN (KNeighborsClassifier (·))	n_neighbors = 5, weights = 'uniform', algorithm = 'auto', leaf_size = 30, p = 2, metric = 'minkowski'
SVM (SVC (·))	C = 1.0, kernel = 'rbf', degree = 3, gamma = 'scale', probability = True, class_weight = None, shrinking = True, max_iter = -1
ANN (MLPClassifier (·))	hidden_layer_sizes=(100) (single hidden layer with 100 neurons), activation = 'relu', solver = 'adam', alpha = 0.0001, batch_size = 'auto', learning_rate_init = 0.001, max_iter = 200, shuffle = True, epsilon = 1e-8
DNN (KerasClassifier (·))	hidden_layers = 3, activation = 'relu', dropout_rate = 0.3, Units (1st hidden layer) = 16, Units (subsequent layers) = 8, Units (output layer) = 1, Output activation = "sigmoid", batch_size = 32, epochs = 30, Loss function = 'binary_crossentropy', Optimizer = 'adam', Metrics = 'accuracy'

models. The trained IDS models' performances are evaluated using the testing set during the testing phase. Fig. 6 illustrates the IDS modeling process. The six chosen algorithms are used to build fifty-four IDS models, which are evaluated using various metrics to assess the effects of feature selection and normalization as well as the effectiveness of the IDS

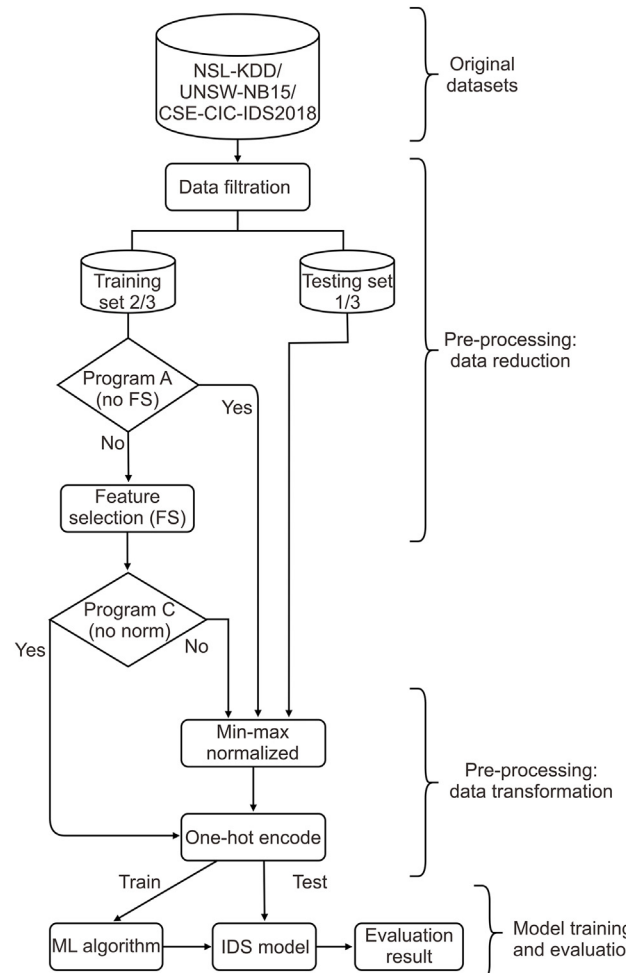


Fig. 6. Intrusion detection system (IDS) modeling process. Note: norm: normalization.

datasets and the selected algorithms in IDS modeling. A summary of the three IDS programs implemented is as follows:

1. Program A: 18 models implemented using the full, non-feature selected, normalized datasets.
2. Program B: 18 models implemented with all the feature-selected, normalized datasets.
3. Program C: 18 models implemented with all the feature-selected, non-normalized datasets.

Algorithm 4 10-Fold cross-validation for IDS model building

```

1: Input: Training dataset  $D_{train}$ , Testing dataset  $D_{test}$ , Algorithms list  $A$  (e.g., [RF, SVM, etc.])
2: Output: Best model with hyperparameters, Performance scores (AccTr, AccPr, DR, FPR), Confusion Matrix, Computational Time
3: Initialize: Best_Model  $\leftarrow$  None, Best_AccTr  $\leftarrow$  0.0, Confusion_Matrix  $\leftarrow$  zeros( $n_{classes}$ ,  $n_{classes}$ ), Computational_Time  $\leftarrow$  0.0
4: for each algorithm  $algo \in A$  do
5:   AccTr_list  $\leftarrow$  [], Time_train_fold  $\leftarrow$  []
6:   for each fold  $i$  in 1 to 10 do
7:      $D_{train\_fold}$ ,  $D_{val\_fold} \leftarrow$  KFoldSplit( $D_{train}$ ,  $n\_splits = 10$ ,  $shuffle = True$ )
8:      $best\_params\_fold \leftarrow$  GridSearchCV( $algo$ ,  $param\_grid$ ,  $scoring = 'accuracy'$ ,  $cv = D_{val\_fold}$ )
9:      $t0 \leftarrow$  time.time()
10:     $model\_fold \leftarrow$  algo(**  $best\_params\_fold$ ).fit( $D_{train\_fold}$ )
11:     $y\_true\_fold$ ,  $y\_pred\_fold \leftarrow D_{val\_fold}.target$ ,  $model\_fold.predict(D_{val\_fold}.data)$ 
12:    AccTr_fold  $\leftarrow$  EvaluateModel( $y\_true\_fold$ ,  $y\_pred\_fold$ )
13:    Time_train_fold  $+=$  (time.time() -  $t0$ )
14:    AccTr_list.append(AccTr_fold)
15:  end for
16:  Avg_AccTr  $\leftarrow$  mean(AccTr_list)
17:  if Avg_AccTr > Best_AccTr then
18:    Best_Model  $\leftarrow$  model_fold
19:    Best_AccTr  $\leftarrow$  Avg_AccTr
20:    Computational_Time  $\leftarrow$  sum(Time_train_fold)
21:  end if
22:  start_time  $\leftarrow$  time.time()
23:   $y\_true$ ,  $y\_pred \leftarrow D_{test}.target$ , Best_Model.predict( $D_{test}.data$ )
24:  AccPr, DR, FPR, Confusion_Matrix  $\leftarrow$  EvaluateModel( $y\_true$ ,  $y\_pred$ )
25:  Computational_Time  $+=$  time.time() - start_time
26: end for
27: Return: Best_Model, (Best_AccTr, AccPr, DR, FPR), Confusion_Matrix, Computational_Time

```

4.5. Model evaluation metrics

The performance of an IDS model can be assessed based on its ability to correctly classify network traffic as normal or abnormal. IDS commonly uses FAR, DR, and accuracy (Khraisat et al., 2019). Similarly, this study uses the same metrics in addition to computation time. The following section explains these metrics and the confusion matrix. The confusion matrix underpins all evaluation metrics in this research, excluding computation time. Further, these metrics facilitate selecting the optimal model.

4.5.1. Confusion matrix

One of the most simple and intuitive metrics for determining a model's accuracy and correctness is the confusion matrix (see Table 10).

Table 10
Confusion matrix.

Class	ID approach	Predicted class	
		Anomaly	Normal
Actual class	Anomaly	TP (Good: correct detection)	FN (Bad: incorrect prediction)
	Normal	FP (Bad: incorrect detection)	TN (Good: correct prediction)

Note: TP: true positive; FP: false positive; FN: false negative; TN: true negative.

It is applied to classification problems where the result may belong to two or more class types (Shobha and Rangaswamy, 2018). Because there are so many implemented models in this work, there is no confusion matrix included.

Basic confusion matrix terminologies:

- True positive (TP): number of attacks correctly detected as an attack.
- False negative (FN): number of attacks incorrectly detected as normal. Aka Type II error.
- False positive (FP): number of normal instances incorrectly detected as an attack. Aka Type I error.
- True negative (TN): number of normal instances correctly detected as normal.

4.5.2. Accuracy (Acc)

Accuracy measures the proportion of correctly classified instances out of the total instances. It is suitable for datasets with symmetric target classes and equal class importance (Shobha and Rangaswamy, 2018). Both training (AccTr) and testing (AccPr) accuracies are reported in this study.

$$Accuracy(Acc) = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

4.5.3. DR

Also known as recall, sensitivity, hit rate, or true positive rate (TPR) is the ratio of accurate positive predictions to the total number of positive predictions. Simply put, it reflects the classifier's sensitivity to detecting positive instances. The higher its value, the better (Shobha and Rangaswamy, 2018).

$$DR = \frac{TP}{TP + FN} \quad (12)$$

4.5.4. FAR

Also known as fall-out or FPR can be defined as the percentage of negative predictions that are mistakenly interpreted as positive (anomaly) for all negative predictions. Better is indicated by a lower value (Shobha and Rangaswamy, 2018).

$$FAR = \frac{FP}{FP + TN} \quad (13)$$

4.5.5. Computational time

The term "computational time" in this work refers to the total time needed to train, assess, and test a model, excluding the feature selection phase. It should be noted that training time is based on the application of 10-fold cross-validation, which can lead to significant computational overhead for numerous algorithms.

5. Results and discussion

This section presents the results and discussion of our study, focusing on various comparisons outlined below. These findings not only highlight relevant insights but also offer managers actionable recommendations for improving their IDS strategies.

- Comparison between the IDS models developed using full datasets and feature-selected datasets to assess the impact of feature selection.
- Comparison between the IDS models developed using normalized and non-normalized feature-selected datasets to assess the impact of normalization.
- Comparison to measure the effectiveness, reliability, and intricacy of the IDS datasets.
- Comparison of the best-performing IDS model with state-of-the-art works to assess the significance of our result.

Table 11
Feature selection (FS) comparison.

Models	Metrics	NSL-KDD		UNSW-NB15		CSE-CIC-IDS2018	
		Full	FS	Full	FS	Full	FS
ANN	AccTr (%)	99.60	98.85	94.50	94.27	95.43	94.85
	AccPr (%)	99.67	98.96	94.36	94.25	95.45	94.71
	DR (%)	99.62	98.68	96.20	97.76	91.40	91.03
	FAR (%)	0.28	0.81	9.57	13.22	0.48	1.60
	Time (ms)	3.35	5.6	12.55	10.10	74.78	29.43
SVM	AccTr (%)	98.53	98.01	93.60	93.55	92.56	89.89
	AccPr (%)	98.56	98.06	93.67	93.46	92.24	89.36
	DR (%)	98.13	97.18	99.63	99.50	92.37	92.77
	FAR (%)	1.08	1.17	19.14	19.44	7.90	14.06
	Time (ms)	23.68	29.91	181.06	307.08	4,373.28	4,637.41
KNN	AccTr (%)	99.52	99.11	93.80	94.62	95.26	95.31
	AccPr (%)	99.57	99.06	93.82	94.76	95.31	95.29
	DR (%)	99.50	99.12	96.26	96.68	93.05	93.01
	FAR (%)	0.36	0.98	11.42	9.36	2.42	2.42
	Time (ms)	2.13	1.89	5.84	5.05	43.19	6.98
RF	AccTr (%)	99.79	99.72	95.84	96.05	95.16	95.15
	AccPr (%)	99.80	99.73	95.78	96.01	95.05	95.12
	DR (%)	99.71	99.68	97.88	97.85	92.93	93.00
	FAR (%)	0.11	0.23	8.73	7.90	2.82	2.76
	Time (ms)	0.32	0.29	0.85	0.63	4.58	2.30
NB	AccTr (%)	85.68	84.59	48.14	48.16	64.53	51.10
	AccPr (%)	85.69	84.81	48.08	47.95	63.50	50.96
	DR (%)	69.50	67.61	23.91	23.56	96.93	2.89
	FAR (%)	0.22	0.22	0.02	0.01	70.06	0.78
	Time (ms)	0.07	0.07	0.09	0.08	0.17	0.09
DNN	AccTr (%)	99.17	98.11	93.62	93.26	94.19	79.29
	AccPr (%)	99.25	98.20	93.58	93.20	91.53	77.38
	DR (%)	99.09	97.06	99.87	94.49	93.62	95.43
	FAR (%)	0.61	0.81	19.94	9.56	10.58	40.75
	Time (ms)	22.93	17.98	28.69	27.64	128.23	106.40

Table 12
Normalization comparison.

Models	Metrics	NSL-KDD		UNSW-NB15		CSE-CIC-IDS2018	
		Norm	Non-norm	Norm	Non-norm	Norm	Non-norm
ANN	AccTr (%)	98.85	95.58	94.27	92.84	94.85	65.87
	AccPr (%)	98.96	96.17	94.25	91.31	94.71	63.24
	DR (%)	98.68	97.48	97.76	96.10	91.03	96.80
	FAR (%)	0.81	4.96	13.22	18.90	1.60	70.45
	Time (ms)	5.60	1.41	10.10	10.06	29.43	22.74
SVM	AccTr (%)	98.01	53.51	93.55	75.49	89.89	55.55
	AccPr (%)	98.06	53.50	93.46	75.40	89.36	54.99
	DR (%)	97.18	0.11	99.50	97.90	92.77	99.00
	FAR (%)	1.17	0.03	19.44	72.60	14.06	89.19
	Time (ms)	29.91	375.34	307.08	932.26	4,637.41	8,390.91
KNN	AccTr (%)	99.11	99.38	94.62	93.64	95.31	94.64
	AccPr (%)	99.06	99.42	95.83	93.66	95.29	94.66
	DR (%)	99.12	99.47	97.35	96.41	93.01	92.20
	FAR (%)	0.98	0.63	7.41	12.19	2.42	2.88
	Time (ms)	1.89	1.82	5.05	4.90	6.98	1.30
RF	AccTr (%)	99.72	99.86	96.05	96.01	95.15	94.84
	AccPr (%)	99.73	99.88	96.01	96.07	95.12	94.82
	DR (%)	99.68	99.79	97.85	97.89	93.00	93.43
	FAR (%)	0.23	0.04	7.90	7.81	2.76	3.78
	Time (ms)	0.29	0.28	0.63	0.62	2.30	2.84
NB	AccTr (%)	84.59	53.51	48.16	52.23	51.10	54.22
	AccPr (%)	84.81	53.41	47.95	52.43	50.96	54.61
	DR (%)	67.61	1.71	23.56	32.38	2.89	96.41
	FAR (%)	0.22	1.57	0.01	4.82	0.78	87.36
	Time (ms)	0.07	0.06	0.08	0.07	0.09	0.09
DNN	AccTr (%)	98.11	96.13	93.26	84.07	79.29	63.40
	AccPr (%)	98.20	96.72	93.20	81.42	77.38	62.17
	DR (%)	97.06	93.49	94.49	99.70	95.43	95.68
	FAR (%)	0.81	0.47	9.56	57.55	40.75	71.47
	Time (ms)	17.98	17.17	27.64	27.89	106.40	105.18

Note: Norm: normalization.

5.1. Feature selection

Table 11 presents the results of models built using the full and selected features of the datasets. Feature selection generally improves performances on new unseen data and reduces computational complexity (Kang and Kim, 2016; Song et al., 2013; Zhou et al., 2020). With NSL-KDD, it can be seen that in comparison to the performance of models built using full features, the models built with 19 selected features achieved similar performances albeit slightly lower (this is consistent with Khammassi and Krichen (2017)). Whereas with UNSW-NB15, the performance of three models (RF, KNN, and NB) improved on the full and the 20 selected features. ANN and SVM models achieved similar performance on the full and selected features. Unlike with NSL-KDD models, the performance of ANN, SVM, KNN, and RF models with CSE-CIC-IDS2018 was similar to UNSW-NB15, except for NB and SVM which performed less on the selected features of CSE-CIC-IDS2018. The NB models achieved the best and lowest computational time whereas SVM achieved the opposite. In all three datasets, DNN models performed better with full features. Furthermore, the CSE-CIC-IDS2018 has higher performance differences between full feature and feature-selected models. Thus, feature selection improves the performances of more models implemented with UNSW-NB15 and CSE-CIC-IDS2018 than with NSL-KDD. Overall, the RF and then KNN benefited the most from the feature selection, with the contrast observed with ANN and DNN models. Thus, while feature selection can be beneficial for simpler models, it may not always lead to improved performance for complex models like ANNs and DNNs, which are designed to handle high-dimensional data and complex feature interactions (Burkov, 2019; Hagan et al., 2014).

5.2. Normalization

Table 12 presents the results of models built using the normalized and non-normalized feature-selected data of the datasets. Normally normalization improves performance and decreases computation time (Kasongo and Sun, 2020b; Wang et al., 2009). The performances of three models (ANN, SVM, and DNN) were all improved by normalization with the SVM benefiting the most as expected (Hsu et al., 2016; Wu et al., 2008). However, both KNN and RF are not much affected by normalization, achieving roughly similar performance across the datasets. NB performance improved after normalization only with NSL-KDD. Overall, normalization has improved the performance of at least 4 classifiers across each dataset. Thus, although normalization's importance in IDS modeling is often ignored, it improves performance. These findings are consistent with Wang et al. (2009), who compared four normalization methods for anomaly intrusion detection using SVM, PCA, and KNN. Furthermore, RF and NB do not necessarily need normalization as explained by Wu et al. (2008) and Lewinson (2020), hence their computational time with and without normalization is low and correspondingly close. Besides, in both normalized and non-normalized versions of each dataset, the NB models achieved the best and lowest computational time whereas SVM achieved the opposite. Overall, the sum of computational time taken by the normalized models is considerably lower than that of the models built with non-normalized data. Thus, normalization also reduces computational time in addition to improving performance.

5.3. Datasets comparison

To assess the reliability and intricacy of the datasets, we examine them from two perspectives. Firstly, a more general observation of the models' performances in Tables 11 and 12, specifically focusing on models whose performances or computation times were rather surprising, or did not show similar effects after performing related actions compared to similar models on corresponding datasets. Secondly, although the feature selection process on both datasets is the same, more features were selected in UNSW-NB15 (20) and NSL-KDD (19) than in

CSE-CIC-IDS2018 (12), so for fair and transparent comparisons, we consider the performances of the models implemented using full and normalized features (Program A) since they are all had the same normalization.

5.3.1. Datasets reliability

Since KNN typically uses distance measures to find k nearest points from any given point, using the normalized features should generally enable all features to be of equal importance thereby improving its performance (Wu et al., 2008). However, while normalization does improve KNN performance with UNSW-NB15 and CSE-CIC-IDS2018, the reverse is seemingly the case with NSL-KDD. As seen in Table 11, the feature selection did reduce the performance of KNN and RF with NSL-KDD; however, with UNSW-NB15 and CSE-CIC-IDS2018, the performances of both KNN and RF across all the metrics were improved. Furthermore, the DNN, SVM, ANN, and then NB in Table 12 showed high FAR on non-normalized CSE-CIC-IDS2018 followed by non-normalized UNSW-NB15; however, with NSL-KDD the opposite is achieved, the high FAR might be due to the intricacies and stealthiness of modern-day attacks traffic in the two datasets which the NSL-KDD lacks.

5.3.2. Datasets intricacy

The performances of models implemented using full and normalized features (Program A) were used for further comparisons on accuracy, DR, and FAR as depicted in Figs. 7–9, respectively.

In Fig. 7, all NSL-KDD models outperform their corresponding UNSW-NB15 and CSE-CIC-IDS2018 models with the highest accuracy of 99.79% and 95.84% achieved by RF for NSL-KDD and UNSW-NB15, respectively and ANN with 95.43% for CSE-CIC-IDS2018. In contrast, NB achieved the worst accuracy in all datasets. Furthermore, in the DR shown in Fig. 8, which stands for the accuracy rate of correctly detecting attack traffic, four of the NSL-KDD models detect more attacks than their corresponding models except for DNN and surprisingly, NB respectively recorded higher DRs of 99.87% and 96.93% on UNSW-NB15 and CSE-CIC-IDS2018 than their related models. RF again achieves the overall highest DR of 99.71% with NSL-KDD. The NB achieved the lowest DR for NSL-KDD (69.5%) and UNSW-NB15 (23.91%) followed by ANN on CSE-CIC-IDS2018 with 91.4%.

The FAR depicted in Fig. 9 shows notable alert rate differences between the NSL-KDD and other datasets' models. Except with NB on UNSW-NB15 which has the lowest FAR of 0.02%, the NSL-KDD models have lower FAR percentages than their corresponding models on different datasets. Surprisingly, NB has the highest FAR over 70% on CSE-CIC-IDS2018. On average, the FAR percentage for all the NSL-KDD models is many times less than for other dataset models.

A high DR and low FARs are generally the targets in IDS (Khraisat et al., 2019). Overall, the prediction accuracy, DR, and FAR of models

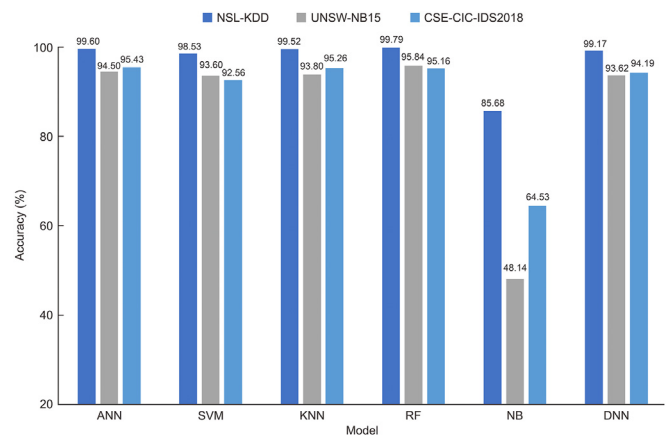


Fig. 7. Program A models accuracy.

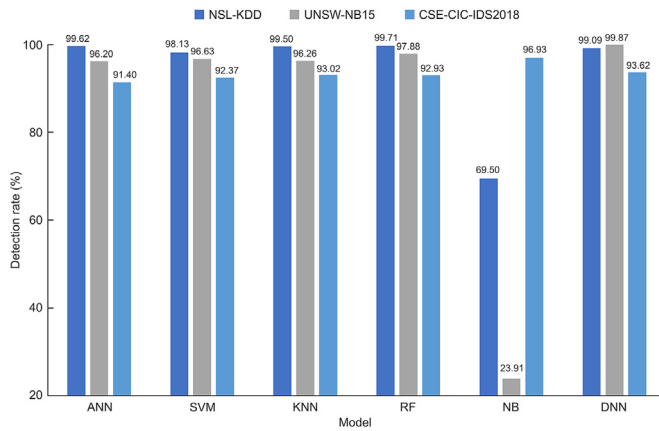


Fig. 8. Program A models detection rate.

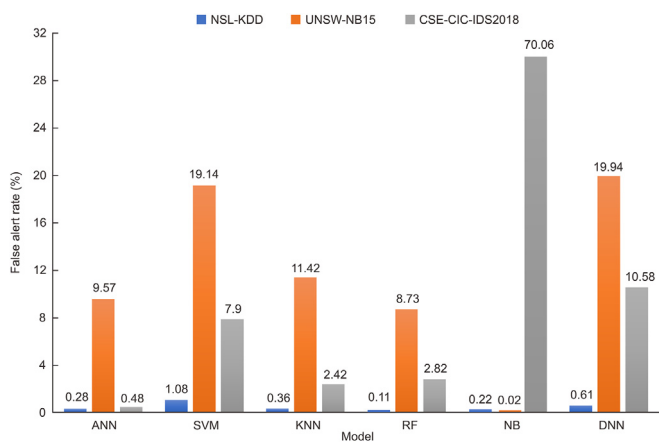


Fig. 9. Program A models false alert rate.

built with NSL-KDD appear to be better than those on different datasets, a typical understanding is that NSL-KDD is indeed better; however, it can be observed that UNSW-NB15 and CSE-CIC-IDS2018, unlike NSL-KDD which has few and outdated attack families, contain different modern low footprint attack families which exhibit similar behavior to normal network traffic, this will essentially make it difficult for many classifiers to detect its attack patterns accurately. This reflects, precisely, the contemporary real-world network traffic scenarios. The current real-world environment is much more challenging than the ones depicted by the old NSL-KDD dataset and thus UNSW-NB15 and CSE-CIC-IDS2018 are more complex and reliable for evaluating modern-day IDSs than NSL-KDD.

5.4. Comparison with similar works

To assess the effectiveness of the models, we compare the best-performing models in each dataset with the recent studies utilizing feature selection and/or similar datasets in the modeling process. The comparison shown in Table 13 was performed based on the major evaluation metrics.

From Table 13, the best FAR is achieved by Sarvari et al. (2020) for NSL-KDD, Khan et al. (2019) for UNSW-NB15, and our model for CSE-CIC-IDS2018 using ANN, soft-max classifier, and ANN, respectively. The low FAR results might have been influenced by the use of neural network-based classifiers which are proving to be good in IDS classification tasks recently (Vinayakumar et al., 2019a, 2019b). The best accuracy of 99.86% with NSL-KDD, 96.01% with UNSW-NB15, and 99.1% with CSE-CIC-IDS2018 were respectively achieved by two of our

proposed RF models and LSTM-AE methods (Hnamte et al., 2023), our RF methods also record the best DR of 99.79% with NSL-KDD. Thakkar and Lohiya (2023) and Hnamte et al. (2023) respectively achieved high DR on UNSW-NB15 with DNN and CSE-CIC-IDS2018 with LSTM-AE. Most of the DL-based methods performed well without utilizing feature selection, this further shows their ability to handle complex data (Burkov, 2019). While there are no significant accuracy and DR performance differences between DL and ML-based models across the three datasets, the DL-based models notably recorded very low FAR compared to other classifiers. With the lowest accuracy of 85.56% and second to-worst FAR (15.78%) achieved by Moustafa and Slay (2016), which is the only work utilizing an ML-based algorithm without feature selection, this comparison further highlights the importance of feature selection when using ML-based algorithm and the vice versa with DL algorithms. While this work might not have achieved a better FAR with some of the datasets, it is important to note that in IDS, not detecting an attack can be more harmful than misclassifying normal traffic (Kaushik and Al-Raweshidy, 2024), thus DR can be more important than FAR and any other metrics, and hence, our method can be considered effective for real-world scenarios. Finally, DL and graph-based solutions are increasingly becoming effective in IDS modeling, as evidenced by their excellent performances.

The results indicate that robust algorithms like RF and complex models such as ANN and DNN, when combined with normalization techniques, can significantly improve IDS performance. These insights are crucial for managers aiming to develop effective and efficient security measures, ensuring high DRs and minimizing false alerts.

6. Managerial implications

This study's findings have several important implications for network security managers and IDS implementation. These implications can assist managers in making informed decisions while also increasing the efficiency and effectiveness of their security measures. These implications can be summarized in six points as follows:

- **Feature selection:** This study found that feature selection benefits simpler models like RF and KNN, but it has no significant effect on the performance of complex models like ANNs and DNNs. Managers can benefit from this by investing in ANN and DNN solutions for high-dimensional data scenarios that do not require extensive feature engineering, thereby simplifying the implementation process.
- **Normalization and computational efficiency:** Normalization significantly improves the performance of certain models, particularly SVM, ANN, and DNN, by improving detection capabilities while minimizing computational time. Managers should incorporate normalization into the preprocessing steps to improve the performance of these models.
- **Algorithm selection:** The study discovered that RF models consistently outperformed other algorithms across a variety of datasets. Managers should prioritize RF when choosing an IDS algorithm because of its robustness and high performance, which ensures a dependable defense mechanism against intrusion.
- **Dataset selection:** The choice of dataset significantly impacts the performance of IDS models. The study suggests using modern datasets like UNSW-NB15 and CSE-CIC-IDS2018 instead of older ones like KDD99 and its variants such as NSL-KDD. These datasets include current attack patterns, making them better suited for evaluating and developing reliable IDS models.
- **Real-world application:** The study highlights the importance of achieving a high DR while minimizing false alerts. Managers should prioritize models that strike a balance between these metrics to ensure effective intrusion detection while reducing false positives. High FAR can overwhelm security analysts, resulting in alert fatigue,

Table 13
State-of-the-art comparisons.

Reference	Dataset	FS	Algorithm	Acc (%)	DR (%)	FAR (%)
Setiawan et al. (2019)	NSL-KDD	Modified-RIGFS	SVM	99.6	99.7	0.566
Abdullah et al. (2018)	NSL-KDD	IG-filters	Voting classifier	86.67	86.7	0.12
Golrang et al. (2020)	NSL-KDD	NSGAI-ANN	RF	99.4	99.4	6.0
Sarvari et al. (2020)	NSL-KDD	MCF	ANN	98.81	97.25	0.02
Thakkar and Lohiya (2023)	NSL-KDD	Statistical importance fusion	DNN	99.84	98.81	1.10
Gupta et al. (2022)	NSL-KDD	HCSGA	DLHNN	99.52	96.78	–
Alazzam et al. (2020)	NSL-KDD	PIO	DT	88.3	86.6	8.80
This work	NSL-KDD	DT wrapper-based	RF	99.86	99.79	0.04
Khan et al. (2019)	UNSW-NB15	DSAE	Soft-max classifier	89.13	–	0.75
Umar et al. (2020)	UNSW-NB15	DT-based	RF	86.41	97.95	27.73
Thakkar and Lohiya (2023)	UNSW-NB15	Statistical importance fusion	DNN	89.03	98.95	1.10
Kasongo and Sun (2020a)	UNSW-NB15	WFEU (ExtraTrees)	FFDNN	87.10	–	–
Golrang et al. (2020)	UNSW-NB15	NSGAI-ANN	RF	94.8	94.8	6.0
Moustafa and Slay (2016)	USWN-NB15	–	DT	85.56	–	15.78
Du et al. (2023)	USWN-NB15	–	CNN-LSTM	94.43	93.5	3.97
This work	UNSW-NB15	DT wrapper-based	RF	96.01	97.89	7.81
Khan (2021)	CSE-CIC-IDS2018	–	CNN-RNN	97.75	–	1.40
Lin et al. (2019)	CSE-CIC-IDS2018	–	LSTM	96.2	96.0	–
Hu et al. (2024)	CSE-CIC-IDS2018	–	ESSAM-BiLSTM	98.1	98.28	–
Hnamte et al. (2023)	CSE-CIC-IDS2018	–	LSTM-AE	99.10	99.10	–
This work	CSE-CIC-IDS2018	–	ANN	95.43	91.04	0.48
Sai Chaitanya Kumar et al. (2024)	CIC-IDS-2017	NBGOA	DCRNN	99.56	99.07	0.78
Lo et al. (2022)	ToN-IoT	–	E-GraphSAGE	97.87	97.86	1.92
Lo et al. (2022)	BoN-IoT	–	E-GraphSAGE	99.99	99.99	0.00

Note: Acc: accuracy.

increased operational costs, and potential system vulnerabilities if the IDS is turned off due to a loss of trust.

- Future planning and policy development: The findings of this study can inform future planning and policy development. Managers can use the findings to create new security policies, allocate resources more effectively, and plan for the deployment of advanced IDS models tailored to their specific network environments.

7. Conclusions

Network and computer systems are continually facing increasing attacks while existing protective mechanisms are failing; thus, more effort should be exerted towards analyzing and improving such protective methods, as well as in developing more sophisticated ones to secure networks and systems and address current challenging environments. This study analyzes the effects of feature selection and normalization techniques on NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018 IDS datasets using six AI algorithms. Accuracy, DR, FAR, and computational time were used to evaluate and compare the models. The following conclusions can be deduced:

- Both normalization and feature selection are important in building effective and efficient IDS models with improved performance and computational time subject to the AI algorithm used. While normalization is vital with algorithms such as ANN, DNN, or SVM (Hsu et al., 2016), feature selection is useful with other algorithms such as RF.
- Generally, RF models achieved remarkably higher performances across all datasets, hence RF is the most robust among the oft-used AI algorithms in IDS. SVM requires normalization and takes higher computational time than the other algorithms. NB models take the lowest time but achieve the lowest average performances, making it the least effective algorithm for modeling IDS in this work and the poorest compared to many supervised AI algorithms (Caruana and Niculescu-Mizil, 2006).
- All the ANN and DNN models achieved better results with the full features of the datasets, this essentially highlights the insignificance of feature selection on neural networks and their ability to handle complex data. Furthermore, graph-based solutions are increasingly becoming the most effective in IDS modeling as evidenced by their excellent performances.

- UNSW-NB15 and CSE-CIC-IDS2018 contained more modern low-footprint attack families and are found to be more complex and suitable for building and evaluating modern-day IDS than NSL-KDD, which contained outdated attack families. Thus, we recommend using UNSW-NB15 and CSE-CIC-IDS2018 modeling reliable IDS models.
- It is interesting to note that most of the reviewed works using methods other than one-hot encoding generally achieve lower performance results compared to this work. Thus, although its effect is not quite clear, the use of the one-hot encoding method certainly influenced the classifiers' performances. In future work, it will be interesting to study the effect of various encoding methods in IDS modeling.

The study's findings provide managers with actionable insights for increasing the efficiency and effectiveness of their security measures. Managers can create and implement more effective IDS strategies by prioritizing robust algorithms such as RF, incorporating normalization, selecting appropriate datasets, and understanding the importance of high DRs and low FARs. Additionally, leveraging complex models such as ANN and DNN can handle high-dimensional data and complex feature interactions, further enhancing intrusion detection capabilities.

8. Limitations and future work

While the overall results have shown great promises and distinctive conclusions, particularly concerning the effects of the preprocessing techniques and the datasets, there exist some limitations and open gaps for future research as follows:

- Dataset and algorithms: Among the many available IDS dataset benchmarks and algorithms, only three datasets are used in this study with six algorithms. More datasets and algorithms could be utilized for broader insights.
- Alternative feature selection and normalization: There are three feature selection methods; the Wrapper method is considered in this study. However, other methods can be explored in future studies. The same wrapper method can also be used in a different approach, such as changing the evaluation algorithm or the search strategy. Similarly, the use of different normalization methods can also be investigated.

- **Multi-classification:** This work primarily focused on the binary classification of normal and attack network traffic; however, IDS datasets typically contain diverse attack types. Thus, multi-classification work can be done to enable further analysis of the effect of preprocessing techniques on distinctive classes of attacks.
- **Reduction of FAR:** A good IDS should have high DRs and low FARs. While the models built with the UNSW-NB15 and CSE-CIC-ID2018 datasets generally achieved higher DR, a higher number of FAR is also observable. Although this might suggest the intricacy of the datasets, further work can be performed to reduce it.
- **Graph-based solutions:** While AI methods (such as ML and DL) excel at detecting cyberattacks by learning patterns from flat data, they struggle with capturing the structural behavior of attacks. Graph structures, however, offer a more abstract and resilient system view, aiding in effective detection (Jiang, 2022). In particular, GNNs have recently shown success in learning from graph-structured data, helping models better understand attack patterns (Bilot et al., 2023), such a new approach could be considered in future studies.

CRedit authorship contribution statement

Mubarak Albarka Umar: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Zhanfang Chen:** Writing – review & editing, Supervision, Resources, Project administration, Investigation, Funding acquisition. **Khaled Shuaib:** Writing – review & editing, Supervision, Resources, Investigation, Funding acquisition. **Yan Liu:** Writing – review & editing, Validation, Supervision, Resources, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that there are no conflicts of interest.

Acknowledgments

The authors are grateful to many anonymous reviewers whose comments and feedback have contributed to and improved this work.

References

- Abdullah, M., Balamash, A., Alshannaq, A., et al., 2018. Enhanced intrusion detection system using feature selection method and ensemble learning algorithms. *Int. J. Comput. Sci. Inf. Secur.* 16 (2), 48–55.
- Alani, M.M., Awad, A.I., Barka, E., 2023. ARP-PROBE: an ARP spoofing detector for Internet of Things networks using explainable deep learning. *Internet of Things* 23 (Oct.), 100861.
- Alazzam, H., Sharieh, A., Sabri, K.E., 2020. A feature selection algorithm for intrusion detection system based on pigeon inspired optimizer. *Expert Syst. Appl.* 148 (Jun.), 113249.
- Albulayhi, K., Abu Al-Haija, Q., Alsuhbany, S.A., et al., 2022. IoT intrusion detection using machine learning with a novel high performing feature selection method. *Appl. Sci.* 12 (10), 5015.
- Alkasasbeh, M., Al-Haj Baddar, S., 2023. Intrusion detection systems: a state-of-the-art taxonomy and survey. *Arabian J. Sci. Eng.* 48 (Aug.), 10021–10064.
- Ayyagari, M.R., Kesswani, N., Kumar, M., et al., 2021. Intrusion detection techniques in network environment: a systematic review. *Wireless Network* 27 (Feb.), 1269–1285.
- Azizjon, M., Jumabek, A., Kim, W., 2020. 1D CNN based network intrusion detection with normalization on imbalanced data. In: 2020 International Conference on Artificial Intelligence in Information and Communication (ICAIC), Fukuoka, Japan, pp. 218–224.
- Biau, G., Scornet, E., 2016. A random forest guided tour. *Test* 25 (Jun.), 197–227.
- Bilot, T., Madhoun, N.E., Agha, K.A., et al., 2023. Graph neural networks for intrusion detection: a survey. *IEEE Access* 11, 49114–49139.
- Bridges, R.A., Glass-Vanderlan, T.R., Iannacone, M.D., et al., 2019. A survey of intrusion detection systems leveraging host data. *ACM Comput. Surv.* 52 (6), 1–35.
- Buczak, A.L., Guven, E., 2016. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Commun. Surv. Tut.* 18 (2), 1153–1176.
- Burkov, A., 2019. *The Hundred-Page Machine Learning Book*. Andriy Burkov, Polen.
- Caruana, R., Niculescu-Mizil, A., 2006. An empirical comparison of supervised learning algorithms. In: *Proceedings of the 23rd International Conference on Machine Learning*. Association for Computing Machinery, New York, USA, pp. 161–168.
- Cerda, P., Varoquaux, G., Kégl, B., 2018. Similarity encoding for learning with dirty categorical variables. *Mach. Learn.* 107 (Sep.), 1477–1494.
- Cheng, Q., Wu, C., Zhou, S., 2021. Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. *IEEE Commun. Lett.* 25 (5), 1564–1567.
- Cortes, C., Vapnik, V., 1995. Support-vector networks. *Mach. Learn.* 20 (Sep.), 273–297.
- Creech, G., Hu, J., 2013. Generation of a new IDS test dataset: time to retire the KDD collection. In: 2013 IEEE Wireless Communications and Networking Conference (WCNC), Shanghai, China, pp. 4487–4492.
- Depren, O., Topallar, M., Anarim, E., et al., 2005. An intelligent intrusion detection system (IDS) for anomaly and misuse detection in computer networks. *Expert Syst. Appl.* 29 (4), 713–722.
- Du, J., Yang, K., Hu, Y., et al., 2023. NIDS-CNNLSTM: network intrusion detection classification model based on deep learning. *IEEE Access* 11 (Mar.), 24808–24821.
- D'hooge, L., 2022. CSE-CIC-IDS2018. <https://www.kaggle.com/datasets/dhoogla/csecicids2018> (Accessed 10 March 2025).
- D'hooge, L., Verkerken, M., Volckaert, B., et al., 2022. Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models. In: Cavallaro, L., Gruss, D., Pellegrino, G., et al. (Eds.), *Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer International Publishing, Cham, pp. 23–41.
- Golrang, A., Golrang, A.M., Yildirim Yayilgan, S., et al., 2020. A novel hybrid IDS based on modified NSGAII-ANN and random forest. *Electronics* 9 (4), 577.
- Gotorlar, H.G., Aghababa, M.P., Bagerzadeh, J., et al., 2015. Improving intrusion detection using a novel normalization method along with the use of harmony search algorithm for feature selection. In: 2015 7th Conference on Information and Knowledge Technology (IKT), Urmia, Iran, pp. 1–6.
- Gupta, S.K., Tripathi, M., Grover, J., 2022. Hybrid optimization and deep learning based intrusion detection system. *Comput. Electr. Eng.* 100 (May), 107876.
- Hagan, M.T., Demuth, H.B., Beale, M.H., et al., 2014. *Neural Network Design*, second ed. Oklahoma State University.
- Haider, W., Hu, J., Slay, J., et al., 2017. Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling. *J. Netw. Comput. Appl.* 87 (Jun.), 185–192.
- Han, J., Pei, J., Tong, H., 2023. *Data mining: concepts and techniques*. Morgan Kaufmann, fourth ed. Elsevier, Amsterdam.
- Hastie, T., Tibshirani, R., Friedman, J., 2009. *The Elements of Statistical Learning*, second ed. Springer, New York.
- Hnamte, V., Nhung-Nguyen, H., Hussain, J., et al., 2023. A novel two-stage deep learning model for network intrusion detection: lstm-AE. *IEEE Access* 11 (Apr.), 37131–37148.
- Hsu, C.W., Chang, C.C., Lin, C.J., 2016. *A Practical Guide to Support Vector Classification*. Hu, X., Meng, X., Liu, S., et al., 2024. An improved algorithm for network intrusion detection based on deep residual networks. *IEEE Access* 12 (May), 66432–66441.
- James, G., Witten, D., Hastie, T., et al., 2021. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, New York.
- Jiang, W., 2022. Graph-based deep learning for communication networks: a survey. *Comput. Commun.* 185 (Mar.), 40–54.
- Jin, R., Breitbart, Y., Muoh, C., 2007. Data discretization unification. In: *Seventh IEEE International Conference on Data Mining (ICDM 2007)*, Omaha, USA, pp. 183–192.
- Kang, S.H., Kim, K.J., 2016. A feature selection approach to find optimal feature subsets for the network intrusion detection system. *Cluster Comput* 19 (Mar.), 325–333.
- Kasongo, S.M., Sun, Y., 2020a. A deep learning method with wrapper based feature extraction for wireless intrusion detection system. *Comput. Secur.* 92 (May), 101752.
- Kasongo, S.M., Sun, Y., 2020b. Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset. *J. Big Data* 7 (Nov.), 105.
- Kaushik, A., Al-Rawashdy, H., 2024. A novel intrusion detection system for internet of things devices and data. *Wirel. Netw.* 30 (Jan.), 285–294.
- Keshk, M., Koroniotis, N., Pham, N., et al., 2023. An explainable deep learning-enabled intrusion detection framework in IoT networks. *Inf. Sci.* 639 (Aug.), 119000.
- Khammassi, C., Krichen, S., 2017. A GA-LR wrapper approach for feature selection in network intrusion detection. *Comput. Secur.* 70 (Sep.), 255–277.
- Khan, M.A., 2021. HCRNNIDS: hybrid convolutional recurrent neural network-based network intrusion detection system. *Processes* 9 (5), 834.
- Khan, F.A., Gumaei, A., Derhab, A., et al., 2019. A novel two-stage deep learning model for efficient network intrusion detection. *IEEE Access* 7 (Feb.), 30373–30385.
- Khrisat, A., Gondal, I., Vamplew, P., et al., 2019. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity* 2 (Jul.), 20.
- Kumar, S., Gupta, S., Arora, S., 2022. A comparative simulation of normalization methods for machine learning-based intrusion detection systems using KDD Cup'99 dataset. *J. Intell. Fuzzy Syst.* 42 (3), 1749–1766.
- Lakshminarayana, D.H., Phillips, J., Tabrizi, N., 2019. A survey of intrusion detection techniques. In: 2019 18th IEEE International Conference on Machine Learning and Applications (ICMLA). Boca Raton, USA, pp. 1122–1129.
- Leevy, J.L., Khoshgoftar, T.M., 2020. A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data. *J. Big Data* 7 (Nov.), 104.
- Lewinson, E., 2020. *Python for Finance Cookbook: over 50 Recipes for Applying Modern Python Libraries to Finance Data Analysis*. Packt, Birmingham, UK.
- Lin, P., Ye, K., Xu, C.Z., 2019. Dynamic network anomaly detection system by using deep learning techniques. In: Da Silva, D., Wang, Q., Zhang, L.J. (Eds.), *Cloud Computing – CLOUD 2019*, 11513. Springer, Cham, pp. 161–176.
- Lo, W.W., Layeghy, S., Sarhan, M., et al., 2022. E-GraphSAGE: a graph neural network based intrusion detection system for IoT. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. Budapest, Hungary, pp. 1–9.
- Moustafa, N., 2015. *The UNSW-NB15 Data Set Description*. <https://research.unsw.edu.au/projects/unswnb15-dataset> (Accessed 10 March 2025).

- Moustafa, N., Slay, J., 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). Canberra, Australia, pp. 1–6.
- Moustafa, N., Slay, J., 2016. The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Information Security. J.: Global Perspect.* 25 (1–3), 18–31.
- Özgür, A., Erdem, H., 2016. A review of KDD99 dataset usage in intrusion detection and machine learning between 2010 and 2015. *PeerJ Preprints. Technical Report* e1954v1.
- Ramírez-Gallego, S., García, S., Mouriño-Talín, H., et al., 2016. Data discretization: taxonomy and big data challenge. *WIREs Data Mining and Knowledge. Discovery* 6 (1), 5–21.
- Rani, A., Kumar, N., Kumar, J., et al., 2022. Machine learning for soil moisture assessment. In: *Deep Learning for Sustainable Agriculture*. Elsevier, pp. 143–168.
- Ring, M., Wunderlich, S., Scheuring, D., et al., 2019. A survey of network-based intrusion detection data sets. *Comput. Secur.* 86 (Sep.), 147–167.
- Sai Chaitanya Kumar, G., Kiran Kumar, R., Parish Venkata Kumar, K., et al., 2024. Deep residual convolutional neural Network: an efficient technique for intrusion detection system. *Expert Syst. Appl.* 238 (Mar.), 121912.
- Salzberg, S.L., 1994. C4.5: programs for machine learning by J. Ross quinlan. Morgan Kaufmann Publishers, Inc., 1993. *Mach. Learn.* 16 (Sep.), 235–240.
- Sarvari, S., Mohd Sani, N.F., Mohd Hanapi, Z., et al., 2020. An efficient anomaly intrusion detection method with feature selection and evolutionary neural network. *IEEE Access* 8 (Apr.), 70651–70663.
- Setiawan, B., Djanali, S., Ahmad, T., 2019. Increasing accuracy and completeness of intrusion detection model using fusion of normalization, feature selection method and support vector machine. *International J. Intell. Eng. Syst.* 12 (Aug.), 378–389.
- Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. SCITEPRESS - Science and Technology Publications, Funchal, Madeira, Portugal, pp. 108–116.
- Shobha, G., Rangaswamy, S., 2018. Machine learning. In: *Handbook of Statistics*. Elsevier, Amsterdam, pp. 197–228.
- Siddique, K., Akhtar, Z., Aslam Khan, F., et al., 2019. KDD cup 99 data sets: a perspective on the role of data sets in network intrusion detection research. *Computer* 52 (2), 41–51.
- Sivatha Sindhu, S.S., Geetha, S., Kannan, A., 2012. Decision tree based light weight intrusion detection using a wrapper approach. *Expert Syst. Appl.* 39 (1), 129–141.
- Somwang, P., Lilakiatsakun, W., 2011. Computer network security based on support vector machine approach. In: 2011 11th International Conference on Control, Automation and Systems, Gyeonggi-do, Korea (South), pp. 155–160.
- Song, J., Zhu, Z., Scully, P., et al., 2013. Selecting features for anomaly intrusion detection: a novel method using fuzzy c means and decision tree classification. In: Wang, G., Ray, I., Feng, D., et al. (Eds.), *Cyberspace Safety and Security*. Springer, Cham, pp. 299–307.
- Sowmya, T., Mary Anita, E., 2023. A comprehensive review of AI based intrusion detection system. *Measurement. Sensors* 28 (Aug.), 100827.
- Surya Prasath, V.B., Abu Alfeilat, H.A., Hassanat, A.B.A., et al., 2017. Effects of distance measure choice on KNN classifier performance-a review. Available at: <https://arxiv.org/pdf/1708.04321#:~:text=This%20review%20attempts%20to%20answer%20this%20question%20through,and%20without%20adding%20di%20erent%20levels%20of%20noise>.
- Taher, K.A., Mohammed Yasin Jisan, B., Rahman, M.M., 2019. Network intrusion detection using supervised machine learning technique with feature selection. In: 2019 International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST). Dhaka, Bangladesh, pp. 643–646.
- Tavallae, M., Bagheri, E., Lu, W., et al., 2009. A detailed analysis of the KDD CUP 99 data set. In: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, Ottawa, Canada, pp. 1–6.
- Thakkar, A., Lohiya, R., 2023. Fusion of statistical importance for feature selection in deep neural network-based intrusion detection system. *Inf. Fusion* 90 (Feb.), 353–363.
- Thaseen, S., Kumar, C.A., 2013. An analysis of supervised tree based classifiers for intrusion detection system. In: 2013 International Conference on Pattern Recognition, Informatics and Mobile Engineering. Salem, India, pp. 294–299.
- Umar, M.A., Zhanfang, C., Liu, Y., 2020. Network intrusion detection using wrapper-based decision tree for feature selection. In: *Proceedings of the 2020 International Conference on Internet Computing for Science and Engineering*, ACM, New York, USA, pp. 5–13.
- Umar, M.A., Chen, Z., Liu, Y., 2021. A hybrid intrusion detection with decision tree for feature selection. *Inf. Secur. Int. J.* 49, 1–20.
- UNB, 2009. ISCX NSL-KDD dataset 2009. <https://www.unb.ca/cic/datasets/nsl.html> (Accessed 10 March 2025).
- UNB, 2018. CSE-CIC-IDS2018 on AWS. <https://www.unb.ca/cic/datasets/ids-2018.html> (Accessed 10 March 2025).
- Vinayakumar, R., Alazab, M., Soman, K.P., et al., 2019a. Deep learning approach for intelligent intrusion detection system. *IEEE Access* 7, 41525–41550.
- Vinayakumar, R., Soman, K.P., Poornachandran, P., et al., 2019b. Application of deep learning architectures for cyber security. In: Hassanien, A.E., Elhoseny, M. (Eds.), *Cybersecurity and Secure Information Systems*. Springer, Cham, pp. 125–160.
- Wang, W., Zhang, X., Gombault, S., et al., 2009. Attribute normalization in network intrusion detection. In: 2009 10th International Symposium on Pervasive Systems, Algorithms, and Networks, pp. 448–453.
- Wang, Y.C., Houn, Y.C., Chen, H.X., et al., 2023. Network anomaly intrusion detection based on deep learning approach. *Sensors* 23 (4), 2171.
- Witten, I.H., Frank, E., Hall, M.A. (Eds.), 2016. *Data Mining: Practical Machine Learning Tools and Techniques*, fourth ed. Morgan Kaufmann, Amsterdam.
- Wu, X., Kumar, V., Ross Quinlan, J., et al., 2008. Top 10 algorithms in data mining. *Knowl. Inf. Syst.* 14 (Jan.), 1–37.
- Yadav, B.P., Ghate, S., Harshavardhan, A., et al., 2020. Text categorization performance examination using machine learning algorithms. *IOP Conf. Ser. Mater. Sci. Eng.* 981, 022044.
- Zhang, Z., 2016. Introduction to machine learning: k-nearest neighbors. *Ann. Transl. Med.* 4 (11), 218–218.
- Zhou, Y., Cheng, G., Jiang, S., et al., 2020. Building an efficient intrusion detection system based on feature selection and ensemble classifier. *Comput. Network.* 174 (Apr.), 107247.